Degree Project in Electrical Engineering

Second cycle, 30 credits

# Design and Stability Analysis of a Neural-network-based Controller

## An Application to Pendubot

**DALIM WAHBY**

# Design and Stability Analysis of a Neural-network-based Controller

## An Application to Pendubot

DALIM WAHBY

Date: January 13, 2025

Supervisors: Dr. Cristian Rojas, Dr. Guillaume Ducard
Examiner: Dr. Johansson Mikael
School of Electrical Engineering and Computer Science
Host organization: I3S/CNRS
Swedish title: Design och Stabilitetsanalys av en Neuronnätbaserad Reglering
Swedish subtitle: En tillämpning på Pendubot

## Abstract

Control theory is fundamental across various domains, including aerospace and robotics, where managing system dynamics effectively is crucial. Model Reference Adaptive Control (MRAC) is a notable technique for handling systems with uncertain dynamics by adaptively updating controller parameters. Traditional MRAC methods typically use gradient descent or Lyapunov-based update laws, but these approaches face challenges when applied to neural network controllers (NNCs) due to their nonlinear characteristics, which complicate verification and stability analysis. This study explores the performance and stability of various adaptive control strategies, namely linear MRACs, self-tuning PID controllers, and PID Neural Network Controllers (PIDNNs), within the context of the Pendubot, an inverted pendulum model. Furthermore, we investigate how to extend linear MRACs to nonlinear MRACs using Lyapunov theory.

Our research addresses two key objectives: 1) the comparative performance and stability of linear MRACs versus alternative adaptive controllers, applied to Pendubot, and 2) the formulation of stable learning algorithms for nonlinear NNCs. Simulations conducted using MATLAB/Simulink and Python reveal that the self-tuning PID controller, although initially exhibiting significant overshoot, learns to control the system. The PID Neural Network Controller (PIDNN) demonstrate an initial oscillatory response but achieve the desired output within approximately 10 seconds. In contrast, the linear MRAC with state feedback shows superior performance with minimal overshoot and precise tracking of the reference model, with the system's weights converging to expected values, highlighting stable learning and control.

Based on the insights gained from developing linear MRACs, we propose a new update law for nonlinear MRACs, specifically for first-order systems and multiple-input, multiple-output (MIMO) systems, and validate this through simulations. This new update law demonstrates the potential effectiveness of the proposed learning algorithms for NNCs but also reveals the limitations related to the need for complete system knowledge. Futhermore, global stability properties of the proposed MIMO update law is dependent on the chosen system. Future work will focus on determining the conditions under which this update law can provide local stability and exploring further improvements to enhance stability assurances. Finally, numerical stability issues of the dervied update laws remain to be addressed.



## Sammanfattning

Reglerteknik är grundläggande inom olika områden, däribland luft- och rymdteknik samt robotik, där effektiv hantering av systemdynamik är avgörande. Modellreferensiell-adaptiv reglering (MRAC) är en framstående teknik för att hantera system med osäkra dynamiska tillstånd genom att adaptivt uppdatera reglerparametrar. Traditionella MRAC-metoder använder vanligtvis gradientnedstigning eller Lyapunov-baserade uppdateringsregler, men dessa metoder stöter på utmaningar när de tillämpas på neurala nätverksreglerare (NNCs) på grund av deras icke-linjära egenskaper, vilket komplicerar verifiering och stabilitetsanalys. Denna studie utforskar prestanda och stabilitet hos olika adaptiva regleringsstrategier—linjära MRACs, självkorrigerande PID-regulatorer och PID-neurala nätverksregu-

latorer (PIDNNs)—i kontexten av Pendubot-systemet, en inverterad pendelmodell. Vidare undersöker vi hur man kan utöka linjära MRACs till icke-linjära MRACs med hjälp av Lyapunov-teori.

Vår forskning tar itu med två centrala frågor: den jämförande prestandan och stabiliteten hos linjära MRACs jämfört med alternativa adaptiva reglerare, samt formuleringen av stabila inlärningsalgoritmer för icke-linjära NNCs. Simulationer som genomförts med MATLAB/Simulink och Python visar att den självkorrigerande PID-regulatorn, även om den initialt uppvisar betydande överskridningar, lär sig att styra systemet. PID-neurala nätverksregulatorer (PIDNN) visar initialt en oscillerande respons men uppnår det önskade utfallet inom ungefär 10 sekunder. I motsats visar den linjära MRAC med tillståndsfeedback överlägsen prestanda med minimala överskridningar och exakt spårning av referensmodellen, där systemets vikter konvergerar till förväntade värden, vilket påvisar stabilt lärande och reglage.

Baserat på insikterna från utvecklingen av linjära MRACs föreslår vi en ny uppdateringslag för ickelinjära MRACs, specifikt för system av första ordningen, och validerar detta genom simulationer. Denna nya uppdateringslag visar den potentiella effektiviteten hos de föreslagna inlärningsalgoritmerna för NNCs men avslöjar också begränsningarna som är relaterade till behovet av fullständigt systemkunnande. Slutligen föreslår vi en MIMO-uppdateringslag för NNCs och diskuterar de insikter som erhållits från utvecklingen av icke-linjära MRACs. Vi påpekar att uppdateringslagen inte garanterar global stabilitet, men den erhåller lokala stabilitetsgarantier. Framtida arbete kommer att fokusera på att bestämma under vilka förhållanden denna uppdateringslag kan erhålla lokal stabilitet och att utforska ytterligare förbättringar för att stärka stabilitetsgarantierna.

## Acknowledgements

I would like to express my sincere gratitude to several individuals whose support and assistance have been invaluable during my studies and the completion of this project.

First and foremost, I would like to thank my supervisor from the I3S laboratory, Dr. Guillaume Ducard, for enabling my internship and taking the time to discuss and provide insightful feedback weekly throughout the course of this project. I would also like to thank Dr. Miakel Johansson for supervising me on this project.

I would also like to thank Alvaro Detailleur for his collaboration, and the countless discussions that greatly enriched my research experience. Working closely with Alvaro has been a pleasure and very inspiring, encouraging me to dive deeper into the world of control theory.

A special thanks goes to my mother, Beate Rühling, my father, Dr. Daniel Wahby, and my sister, Sina Wahby, whose support has been a source of strength throughout all of my studies. To my friends, thank you for your unwavering support, understanding, and encouragement.

Special thanks to Philipp Ahrendt for sharing the experience of the master's degree in both France and Sweden. I really enjoyed our countless discussions, time working on projects, and studying for exams.

Lastly, I would like to thank everyone who has, in one way or another, contributed to the completion of this thesis. Your support is deeply appreciated.

# Contents

**1 Introduction** **6**
1.1 Introduction . . . 6
1.2 Research Questions . . . 7
1.3 Outline . . . 7
1.4 Ethics and Sustainability . . . 8

**2 Foundations** **9**
2.1 Definition of Systems . . . 9
2.2 Nonlinear Dynamic Inversion . . . 10
2.3 Stability Definition . . . 10
2.4 Control Design . . . 12
2.4.1 PID Control . . . 12
2.4.2 Linear State Feedback Control . . . 13
2.4.3 Linear Quadratic Regulator (LQR) . . . 13
2.5 Model Reference Control . . . 14
2.6 Neural Networks . . . 14

**3 Related Work** **16**
3.1 Brief History on Adaptive Control . . . 16
3.2 Model Reference Adaptive Control (MRAC) . . . 16
3.3 Neural Network Controllers . . . 18
3.4 Noteworthy works on NNC stability . . . 19
3.5 Inverted Pendulums . . . 19
3.6 Contribution . . . 20

**4 Methodology** **21**
4.1 System . . . 21
4.1.1 Inverted Pendulum, called Pendubot . . . 21
4.1.2 Nonlinear Dynamic Inversion (NDI) . . . 21
4.2 Baseline Controllers . . . 22
4.2.1 Self-tuning PID Controller . . . 22
4.2.2 PIDNN with Activation Function . . . 24
4.3 Model Reference Adaptive Control . . . 25
4.3.1 MRACs for State Feedback Control . . . 25
4.3.2 Nonlinear MRACs for First-Order Systems . . . 28
4.3.3 Nonlinear MRACs for Multiple-Input, Multiple-Output Systems . . . 31

**5 Results** **34**
5.1 Baseline Controllers . . . 34
5.1.1 Self-Tuning PID . . . 34

5.1.2 PID Neural Networks for Pendubot . . . 36
5.2 MRAC for Linear State Feedback Control . . . 37
5.3 MRACs for Nonlinear Neural Network Controllers . . . 39
5.3.1 Constant Reference Signal . . . 39
5.3.2 Sine Signal . . . 40
5.3.3 Smooth Pulse Signal . . . 42
5.3.4 Simulation Nonlinear MIMO Systems . . . 42

**6 Discussion & Conclusion 46**
6.1 Discussion . . . 46
6.2 Conclusion . . . 48
6.3 Outlook . . . 49
6.3.1 Partially Uknown Dynamics . . . 49
6.3.2 Investigating the Decrease Condition . . . 49
6.3.3 Mitigation of Steady-State Errors for the Nonlinear MRACs . . . 49
6.3.4 Tackling Numerical Instability . . . 49
6.3.5 Implementation on Hardware . . . 50
6.3.6 Approximating Model Predictive Controllers . . . 50

# 1 Introduction

## 1.1 Introduction

Control theory is a multidisciplinary field that plays a vital role in various domains, including but not limited to aerospace engineering [42], finance [53], and robotics [24]. At its core, control theory is concerned with manipulating an input signal to a system to achieve desired, pre-defined outcomes and behaviors. We call the function that we feed to the system a controller since it *controls* the system at hand.

To ensure that the controlled system follows the desired behavior we can use a technique called model reference control. This technique equates the desired output with the equations of the controlled, closed-loop system, which helps us find the gains for our controller. However, this only works if the system's dynamics are known exactly, which is often far from reality. If we do not have full knowledge of our system, we have to resort to adaptive schematics, which we call model reference adaptive control or MRAC. In MRAC we define a rule with which we update the controller's parameters until we find the optimal gains. The adaptation law can be chosen in two main ways, firstly a gradient descent-based update law as first proposed by Whitaker et al. [66] and applied in for example [9, 10], and secondly, a Lyapunov based update law as proposed by Shackcloth et al. [54]. The latter enforces that in every update step, we have a stabilizing controller, which is not guaranteed when using the former.

Since we do not know the exact controller, we need to find a parametric model that is rich enough to define our control law. For this case, neural networks (NN) are an ideal choice, since they are considered to be universal approximators [23]. Since at least the 90s, research has been conducted on neural network controllers (NNCs) [27], and it has been shown that they can learn a desired behavior, for example in [67, 13, 62, 48]. However, standard NNCs are nonlinear, through the use of nonlinear activation functions. This makes the formal verification of stability more challenging. Often these controllers rely on a posteriori stability verification, which can become non-trivial or is not enough if we require online, stable learning techniques.

To ensure that a system engages in the desired behavior, we need to assess its stability, meaning we need to mathematically analyze and verify if a system converges to the desired output, given the input signal. If this is the case, we say the system is stable. While classical control theory primarily deals with linear time-invariant (LTI) systems, following the superposition principle, real-world systems often exhibit nonlinear behavior. This extends the scope of traditional control theory to handle these more complex, nonlinear systems, which leads us to not be able to use classical stability concepts, such as gain and phase margins. Hence, we need to use more generalized methods, such as Lyapunov theory, which aims to quantify dissipative energy in the system. However, finding an appropriate Lyapunov function can be challenging. When using MRACs with a Lyapunov-based update law, we can impose a Lyapunov function and ensure that the weights satisfy stability conditions on this function. In the MRAC literature however, this is only done for adaptive weights that appear in the form of a linear combination with the error dynamics of the system, meaning the learned parameters are not inside the nonlinear function. Hence, in this work we aim to extend on the MRAC principle and define a nonlinear update rule for MRAC controllers, where the estimated parameters are in a function that

does not satisfy the superposition principle.

To this end, we first investigate the performance of stable linear MRACs compared to existing, not necessarily stable learning schematics, such as a PIDNN and a self-tuning PID controller, applied to an inverted pendulum, called Pendubot. With our insights on developing stable linear MRACs, we go ahead and tackle the problem of stable nonlinear MRACs. For reasons of simplicity, we begin by defining an update law for a first-order system and subsequently propose a MIMO update law. Both newly proposed update laws will are validated through simulations.

## 1.2 Research Questions

We develop stable adaptive control schematics, inspired by Lyapunov theory, for systems controlled by NNCs. Our investigation starts with developing a linear MRAC, which can be considered an NNC with linear activation functions. To this end we derive a stable update law for Pendubot, based on [39] and [6]. Furthermore, we apply the proposed PIDNN method from [62, 10] to Pendubot and compare their performances to a self-tuning PID from [9]. It is noteworthy that the two latter learning schematics are not provably stable. After comparing the three aforementioned approaches, we extend the Lyapunov-based, linear MRAC update law to a Lyapunov-based nonlinear MRAC update law for the simpler scalar case and then a not so trivial MIMO case. More specifically, at the end of this thesis, we will have answered the following research questions:

RQ1 How do linear MRACs compare to self-tuning PIDs, and PIDNNs, applied to Pendubot, with respect to performance and stability?

RQ2 How can we develop stable learning algorithms, based on linear MRACs, for systems controlled by nonlinear NNCs?

## 1.3 Outline

This thesis is organized to provide a comprehensive overview of stable, online learning strategies for MRACs in the linear case, and we propose update schematics for the nonlinear case for NNCs. We commence with foundational concepts in Section 2, where definitions of key terms and various control design strategies, including PID, Linear State Feedback, LQR, Model Reference Control, and Neural Networks, are introduced. Section 3 reviews related work, focusing on adaptive control, MRAC, neural network control, significant stability concepts, and identifying research gaps. The methodology is detailed in Section 4, outlining the system and its control through nonlinear dynamics inversion (NDI), baseline controllers, such as self-tuning PID and PIDNN, and MRAC applied to both linear state feedback and nonlinear first-order systems as well as nonlinear MIMO systems. Section 5 presents the simulation results of these approaches. The thesis is concluded by the discussion and conclusion in Section 6, where we discuss the simulation results.

## 1.4 Ethics and Sustainability

Since developing ethical artificial intelligence is in the interest of the public and sustainability has to be taken into account, since global warming is a great threat to humanity, here in this section we will briefly point out ethical and sustainability considerations of this work.

Ethically, it is crucial to ensure safety and reliability by incorporating robust fail-safe mechanisms to prevent accidents. Furthermore, transparency and accountability must be maintained by ensuring that artificial intelligence decision-making processes are explainable. This work directly aims to this point, through investigating online learning strategies for adaptive controllers and NNCs, that are provably stable, hence pushing research to investigate the exact functionality of black-box NN models.

From a sustainability perspective, optimizing energy efficiency in both computation and physical actuation is vital. Additionally, ensuring long-term reliability through durable design and scalability, while minimizing environmental impact, is important. By focusing on these aspects, the development of control systems not only advances technical knowledge but also promotes responsible and sustainable engineering practices.

From an application point of view, this research can be used in a multitude of domains. For example, if these control strategies are deployed on unmanned aerial vehicles (UAVs), they can be helpful in, including but not limited to environmental sensing, wildlife monitoring, and coastal erosion monitoring. Additionally, these control strategies can be used to facilitate search and rescue missions in remote areas or in situations where it might be dangerous for people to go, or simply to accompany search and rescue teams, to increase the team's safety.

# 2 Foundations

In this chapter, we introduce the necessary concepts to understand the main part of the thesis. We begin by talking about different types of systems and stability concepts. We then move on to Neural Networks (NN) and optimal control strategies.

## 2.1 Definition of Systems

Throughout this work, we consider two types of systems, namely linear time-invariant systems (LTI) and nonlinear systems. We will briefly define them and explain their differences.

Firstly, we consider LTI systems. As their name indicates, they are characterized by linearity and time-invariance, meaning their output is directly proportional to its input and their behavior does not change over time. These properties make LTI systems easier to analyze and predict, as they can be described by constant coefficients, as defined in 2.1 for the continuous case and in 2.2 for the discrete case, and their responses can be computed using convolution and superposition.

**Definition 2.1.1** (Continuous Linear time-invariant system)**.** We define a continuous linear time-invariant (LTI) system as follows

$$\begin{aligned} \dot{x} &= Ax + Bu \\ y &= Cx + Du \end{aligned} \tag{2.1}$$

**Definition 2.1.2** (Discrete Linear time-invariant system)**.** We define a discrete linear time-invariant (LTI) system as follows

$$\begin{aligned} x_{k+1} &= Ax_k + Bu_k \\ y_k &= Cx_k + Du_k \end{aligned} \tag{2.2}$$

Here $A, B, C$, and $D$ are constant matrices. Furthermore, we define the state vector $x \in \mathbb{R}^n$, system matrix $A \in \mathbb{R}^{n\times n}$, input matrix $B \in \mathbb{R}^{n\times p}$, output matrix $C \in \mathbb{R}^{q\times n}$, feed-through matrix $D \in \mathbb{R}^{q\times p}$. Equations 2.1 and 2.2 are also called the state-space representation of a system. In this work, we consider a system with fully accessible states. This results in the $y_k = x_k$, hence we will take only the dynamics concerning the state into account.

In contrast to LTI systems, nonlinear systems are systems which do not comply with the superposition principle. These systems exhibit complex dynamics, including but not limited to multiple equilibria, and can be significantly more challenging to analyze and control compared to linear systems. Hence, we define nonlinear systems more generically, as described in 2.3.

**Definition 2.1.3** (Nonlinear System)**.** We consider a continuous-time nonlinear system $f_c : \mathbb{R}^n \to \mathbb{R}^n$, with its differential equation $\dot{x} = f_c(x(t), u(t))$. We define the associated discrete-time nonlinear system

$$x_{k+1} = f(x_k, u_k) \tag{2.3}$$

Both the definition for LTI and nonlinear systems involve an external input $u_k$, of time step $k$. However, in the scope of this thesis, we aim to develop autonomous systems, meaning that we aim

to define the control law $u$ as a function of the system's state. One way of achieving this is called linear state feedback, which will be defined later on in this work. We also call an autonomous system a *closed loop system.*

**Definition 2.1.4** (Autonomous System)**.** A system that is only dependent on its state and not on an external input, is called an *autonomous system.* Therefore, we say a discrete-time autonomous system is described by

$$f : \mathbb{R}^n \to \mathbb{R}^n, \quad x_{k+1} = f(x_k) \tag{2.4}$$

Specifically, we define an autonomous linear time-invariant system, which has the form as defined in Equation 2.5.

$$\dot{x} = Ax \tag{2.5}$$

## 2.2 Nonlinear Dynamic Inversion

Often, the dynamics of a real-life system are nonlinear. Due to this, designing a controller becomes non-trivial. However, there are ways to cope with these nonlinearities, such as nonlinear dynamic inversion (NDI). It is used to transform a nonlinear dynamic system into an LTI one by applying an appropriate feedback law, simplifying the control problem, as defined in 2.2.1. Later on, we will use NDI to transform our nonlinear system into an LTI system, to simplify the handling of the nonlinearities of our system at hand.

**Definition 2.2.1** (Nonlinear Dynamic Inversion (NDI))**.** Consider the nonlinear plant $\dot{x} = f(x)+g(u)$. We eliminate the nonlinearities by defining the input signal to the nonlinear system as, where $\dot{x}_{des}$ defines the desired reference signal:

$$u = g^{-1}(\dot{x}_{des} - f(x))$$

*Proof.* Consider the system

$$\dot{x} = f(x) + g(u).$$

Using the control input as defined in Definition 2.2.1, hence $u = g^{-1}(\dot{x}_{des} - f(x))$, we get the following Equation:

$$\dot{x}_{des} = f(x) + g(g^{-1}(\dot{x}_{des} - f(x))) = f(x) + \dot{x}_{des} - f(x) = \dot{x}_{des}$$

□

## 2.3 Stability Definition

To analyze the stability of a system, we now define the general concept of stability. We commence by defining equilibrium points, followed by different stability concepts. These directly lead us to the Lyapunov stability criterion, which will be the basis of our notion of stability, with which we will

define the stability of the systems we are taking into account. The introduced notions of stability are taken from [29].

**Definition 2.3.1** (Equilibrium Point)**.** Consider a nonlinear, autonomous system, as defined in Definition 2.1.4. We say that $x^{eq} \in \mathbb{R}^n$ is an equilibrium point of the system, if $x^{eq} = f(x^{eq})$.

**Definition 2.3.2** (Stability Concepts)**.** An equilibrium $x^{eq}$ of 2.1.4 is

- *stable*, if for every $\varepsilon \in (0, \varepsilon_{max}]$, there exists a $\delta > 0$ such that

$$||x_0 - x^{eq}|| \Rightarrow ||x_t - x^{eq}|| \leq \varepsilon, \quad \forall t \geq 0$$

- *asymptotically stable*, if it is stable and $\delta$ can be chosen such that

$$||x_0 - x^{eq}|| < \delta \Rightarrow \lim_{t \to \infty} x_t = x^{eq}$$

- *unstable* otherwise.

The region, from which the system converges towards an equilibrium point is called the region of attraction. Furthermore, a system is asymptotically stable if it is stable and additionally, the limit of the state is the equilibrium point itself. Both concepts are visualized in Figure 2.1, taken from the lecture notes *Linear Quadratic and Model Predictive Control* held by Mikael Johansson [29]. In all other cases, the system is called unstable.

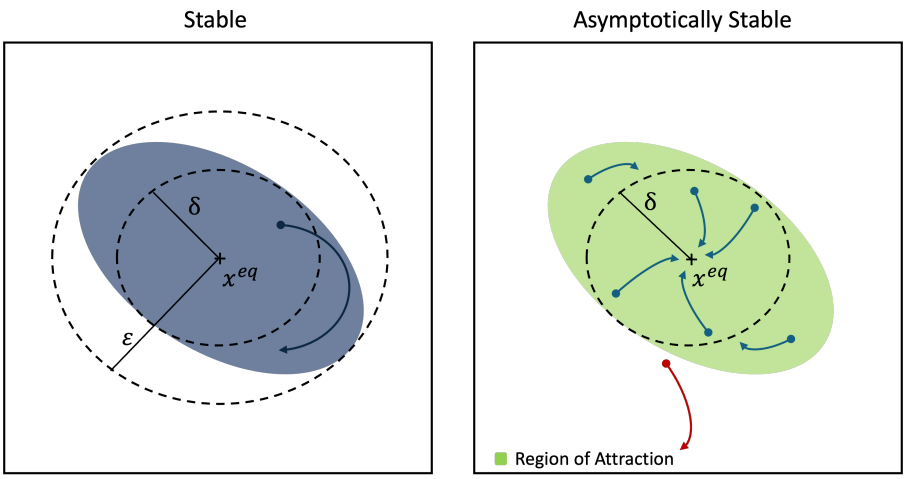



Figure 2.1: Stability and asymptotic stability

Based on the defined concepts of stability, we now define Lyapunov stability. The basic idea of Lyapunov stability is to introduce a positive energy measure and show that it decreases along the system trajectories.

**Definition 2.3.3** (Lyapunov Stability)**.** If there exists a continuous function $V : \mathbb{R}^n \to \mathbb{R}$ such that

(a) $V(x)$ is positive definite, and

(b) $\dot{V}(x) \leq -l(x)$, for some positive semidefinite function $l(x)$,

then the system is considered to be stable. Additionally, if $l(x)$ is positive definite, then the system is asymptotically stable. The function $V$ is called a Lyapunov function.

This definition holds for the continuous case. Respectively, if we consider a discrete system, the condition (b) is swapped with the corresponding difference equation, i.e. $V(x_{k+1}) - V(x_k)$.

A special case, that we will encounter throughout this work is the case of an autonomous LTI system, as defined in Equation 2.5. We consider the Lyapunov candidate $V(x) = x^T P x$, which is positive definite, if and only if $P$ is positive definite. To analyze how the function behaves over time, we find the time derivative and find a condition for stability.

$$\dot{V}(x) = \dot{x}^T P x + x^T P \dot{x} = (Ax)^T P x + x^T P A x = x^T \underbrace{(A^T P + PA)}_{\leq Q} x \tag{2.6}$$

For the system to be stable according to Definition 2.3.3, we require the following linear matrix inequality 2.7 to be satisfied, for some positive definite matrix $Q$. The discrete-time stability criterion can be found by using the difference equation.

$$A^T P + PA - Q \leq 0. \tag{2.7}$$

## 2.4 Control Design

In the context of this thesis, we consider 1) PID controllers, 2) linear state feedback controllers, and the 3) linear quadratic regulator (LQR). The notation for PID controllers is taken from the course Digital Control, held by Dr. Guillaume Ducard [17] and the notation for linear state feedback and LQR is taken from the course on Model Predictive Control, lectured by Dr. Mikael Johansson in [29].

### 2.4.1 PID Control

A PID controller (Proportional-Integral-Derivative controller) is a versatile control loop feedback mechanism. It continuously computes an error as the difference between a desired output and the output of the system, which is followed by applying a corrective action based on three distinct terms, proportional, integral, and derivate terms. The proportional term corrects proportional to the current error, facilitating rapid response but potentially causing an overshoot. The integral term accumulates past errors, addressing any persistent, steady-state error and thus eliminating offset. The derivative term anticipates future error based on its rate of change, thereby contributing to correcting the potential overshoot, enhancing system stability, and reducing oscillatory behavior. The errors are defined as in Equation 2.8. When considering the discrete-time errors, the integral is replaced by summing the errors over all time steps, and the derivate is substituted by the resulting difference equation of the current error and past error, taking into account the sampling rate. These definitions will be used by the adaptive PID controller and the PIDNN throughout this work.

$$e_p = x_r - x, \qquad e_i = \int_0^t e_p, \qquad e_d = \frac{\partial e_p}{\partial t} \tag{2.8}$$

The resulting controller output is defined by Equation 2.9. In the case of an adaptive PID or a PIDNN, the gains $k_p$, $k_i$, $k_d$ are adapted according to an update law that will be defined later on in this work.

$$u = k_p e_p + k_i e_i + k_d e_d \tag{2.9}$$

### 2.4.2 Linear State Feedback Control

A linear state feedback controller is a simple, but very powerful idea. It relies on the idea, that we close the loop of an open loop system and feedback the state to the system, hence the name *state feedback*. Normally, this is done with the opposite sign, meaning that the input to the system will always *steer* in the opposite direction. For example, if the system state lies under the desired state, we feed a positive energy to the system and vice versa. Linear state feedback control enables us to conduct reference tracking, through taking into account the difference between the desired state $x_r$ and the state of the system $x$.

**Definition 2.4.1** (Linear State Feeback Controller)**.** We consider the system as in 2.2. In linear state feedback control, we define $u = -L\Delta x_k$. This results in the following closed-loop system:

$$x_{k+1} = Ax_k + BL\Delta x_k = (A - BL)x_k + BLx_r \tag{2.10}$$

The matrix $L$ is called the feedback matrix.

### 2.4.3 Linear Quadratic Regulator (LQR)

The LQR is an important control design methodology for linear systems because the solution comes with stability and optimality guarantees. We distinguish between an finite horizon and a infinite horizon LQR. The solution to the finite horizon LQR possibly has different solutions, depending on the length of the horizon, whereas the infinite horizon LQR has a stationary and unique solution. This results in us being able to compute the control law a priori. We choose an infinite horizon LQR controller because it is simpler to solve, hence, from here on we only consider the infinite horizon LQR.

**Definition 2.4.2** (Linear Quadratic Regulator)**.** The linear quadratic control problem considers the system in 2.2 and aims to find the control sequence $\{u_0, u_1, u_2, ...\}$, that minimizes the quadratic cost function. The optimization problem is defined as follows:

$$\begin{aligned} \min_u \quad & \sum_{k=0}^{\infty} \Delta x_k^T Q \Delta x_k + \Delta u_k^T R \Delta u_k \\ s.t. \quad & x^+ = Ax + Bu \end{aligned} \tag{2.11}$$

where $\Delta x_k = x_r - x$, $\Delta u_k = u_r - u$ of time step $k$, $Q$ is penalization on state divergence, and $R$ is the penalization on the control input divergence.

Since the cost function is quadratic and both the state space and control input are unconstrained, we can solve the problem using the completion of squares lemma, as presented in the appendix of [29]. This leads to the optimal control static linear feedback control law $u_k = -Lx_k$, with

$$L = (R + B^T PB)^{-1} B^T PA. \tag{2.12}$$

Furthermore, $P$ is the positive semidefinite solution to the Discrete Algebraic Riccati Equation (DARE):

$$P = Q + A^T PA - A^T PB(R + B^T PB)^{-1} B^T PA \tag{2.13}$$

## 2.5 Model Reference Control

We will briefly define model reference control since it is an important concept in the context of MRACs. In model reference control, we aim to design a controller that exhibits the desired behavior of the model reference. In practice, we equate the output of the system with the output of the reference model, to find the form of the controller. The general schematic of a model reference controller can be found in Figure 2.2. This technique will come in handy while defining our model reference adaptive controller (MRAC) since an MRAC is a type of model reference controller, where the controller has a component that can change over time.

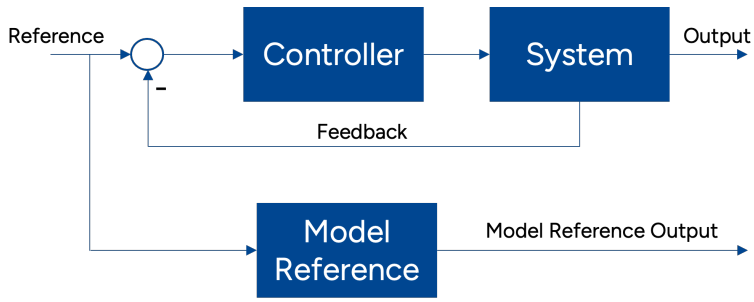



Figure 2.2: Model reference control schematic

## 2.6 Neural Networks

Generally speaking, a neural network (NN) is a parameterized function, that can be adapted to approximate any nonlinear function, as long as it has a sufficient amount of neurons and weights [23]. We aim to use NNs to develop a stable controller for the system at hand. From here onwards, we use the terms NN and NNC interchangeably, since the function defined by the NN will is used as a controller.

**Definition 2.6.1** (Neural network)**.** A neural network (NN) $\phi : \mathbb{R}^n \to \mathbb{R}^p$ is defined as:

$$\begin{aligned} \phi(x) &= (L_1 \circ \varphi_1 \circ L_2 \circ \varphi_2 \cdots \circ L_{H-1} \circ \varphi_{H-1} \circ L_H \circ \varphi_H)(x) \\ L_i(x) &= W_i x + b_i \quad \forall i \in \{1, ..., H\}, \end{aligned} \tag{2.14}$$

where $\varphi_i(\cdot)$ are called activation functions, $W_i$ and $b_i$ are the weight matrix and bias of layer $i$, respectively.

This definition implies, that the input of a layer is defined by the output of the one before. Each neuron has an activation function, which includes but is not limited to linear functions and sigmoid functions. A general schematic of neural networks is given in Figure 2.3.

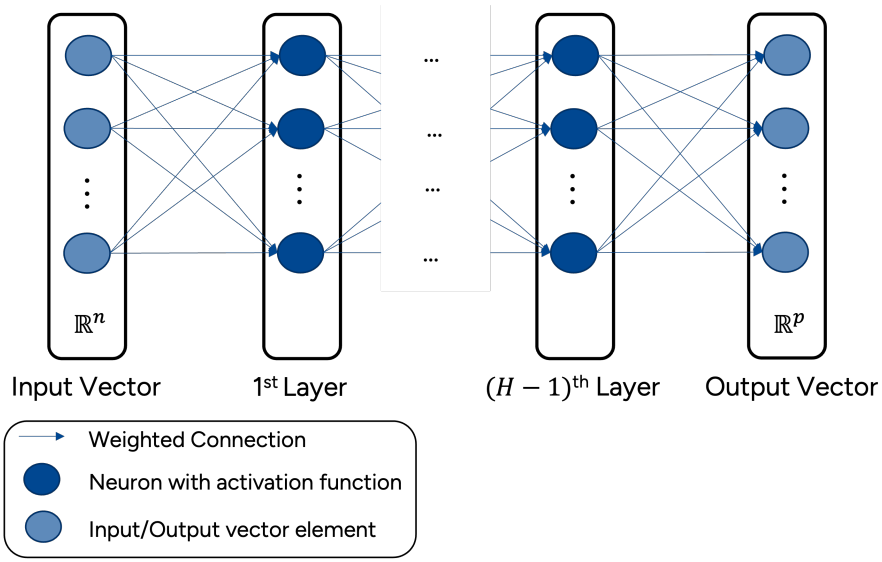



Figure 2.3: General setup of a neural network

The parameters of the NN that are updated are the weights $W_i$ sometimes also $\theta_i$, and the bias $b_i$. Whenever we do not mention a bias, it is assumed to be zero and is not an adaptable parameter. The NNs throughout this work can take different forms. It will be mentioned, whenever we consider a function to be a NN. Furthermore, we mainly use sigmoid activation funtions, which we will denote as $\sigma(\cdot)$.

The parameters of the NN are updated according to an update law that will be defined later on. Typically, a gradient descent-based update law is used, however, this might result in non-bounded weights, possibly leading to instability of the controlled system [1, 43]. Hence, we aim to define stable update rules based on Lyapunov theory later on in this work.

# 3 Related Work

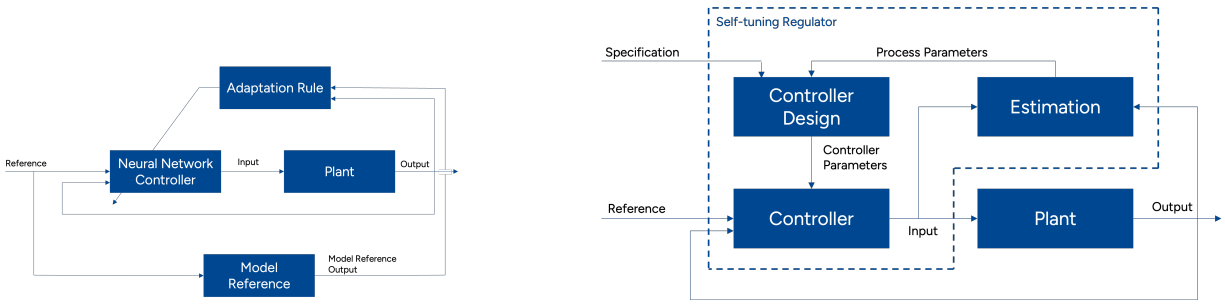
In this chapter, we will talk about related work, relevant to this thesis. We commence with a brief history of adaptive control. Subsequently, we will introduce different types of model reference adaptive controllers or MRACs, which will be followed by approaches using neural network controllers (NNCs). This chapter will conclude with our concrete contributions.

## 3.1 Brief History on Adaptive Control

Classical controllers are usually not well-equipped to deal with process changes or disturbance dynamics. This makes it difficult to control systems that are not easily modeled. To this end, adaptive control was developed. It is a branch of control theory, that deals with controllers that have the ability to change or adapt their parameters over time, according to pre-defined criteria [5].

Adaptive control has been discussed heavily in various forms since its first occurrences in the 1950s and 1960s when research mostly focussed on flight control applications. During this time, which Åstrom calls the brave era [4], guaranteed stability properties were not necessarily one of the most important aspects to consider. A method called the MIT rule was frequently used, even though it was known to not guarantee stability [1, 43]. This resulted in an accidental crash of a rocket, leaving the pilot dead, due to using a partially unstable adaptive controller in 1967 [49].

After the fatal incident, research funding for adaptive control was temporarily suspended. However, in the 1970s, with the rise of two techniques, called Model Reference Adaptive Control (MRAC) and the Self-Tuning Regulator (STR), interest resurged [4]. MRAC is a technique where the gains of a controller are adapted in order to mimic the behavior of a model reference and STR is a technique based on adjusting the controller parameter through an external loop that estimates the parameters, as shown in Figure 3.1. In this thesis, we will put a special focus on MRACs.




(a) MRAC (b) STR

Figure 3.1: MRAC and STR Blockdiagrams [6]

## 3.2 Model Reference Adaptive Control (MRAC)

Model Reference Adaptive Control (MRAC) was first introduced in the early 1970s by Whitaker et al. [65]. Originally, it was thought to deal with process uncertainties and disturbance dynamics, however,

the proposed techniques were also used in different contexts, including but not limited to auto-tuning, automatic construction of gain schedules, and adaptive filtering [5, 4].

The adaptation mechanism of the controller parameters follows two main approaches, namely 1) the MIT method, and 2) the Lyapunov method. The MIT method is an adaptation algorithm that is based on minimizing a cost function, based on the error of the real output compared to the output of the model reference. This adaptation algorithm is called the MIT method or rule [6] but is also known as the gradient descent algorithm. One major drawback of this adaptation technique is, that it does not come with proven stability properties. Mareels et al. discovered and analyzed three main instability mechanisms that accompany the algorithm, namely 1) large adaptation gains, 2) resonance effects, and 3) model mismatch [43]. The Lyapunov method is based on an adaptation rule derived from Lyapunov's second method [54]. It imposes stability since the adaptation rule is chosen in a way that the time derivative of the Lyapunov function is guaranteed to be negative over all time steps, implying convergence of the Lyapunov function itself. However, this approach has the limitation, that one has to choose the adaptation rule in a way that stability is enforced. Here the choice of the Lyapunov function can be challenging.

Generally, MRACs can be split into three categories, firstly direct and secondly indirect MRAC. The former aims to adapt controller parameters directly and the latter aims to update the model parameters. The third category is a hybrid approach called *Combined MRAC*, first proposed by Duarte et al. in [14] for first-order systems. The approach is based on estimating two parts with two separate adaptation rules, namely 1) unmatched model uncertainty and 2) system parameters. Their approach showed in simulations to be more robust than both direct and indirect adaptive control if considered individually [46], however, this has yet to be proven. Combined MRACs are generalizable for $n$-th order linear systems, as proposed by Lavretsky and Tao et al. [39, 61].

Simultaneously, Slotine et al. developed a similar approach, called *Composite MRAC* [59, 58, 57], extending MRACs to a class of nonlinear systems, where the estimated parameters appear linearly in the known, nonlinear dynamics of the system. Applications of the approaches proposed by Narendra et al. and Slotine et al. have shown to be effective and include but are not limited to [15, 16].

In 2000, Patiño et al. proposed to leverage indirect MRACs to adaptively compensate for nonlinearities in the plant [51]. They propose to use Radial Basis Functions (RBF), whose parameters are adapted, to compensate for nonlinearities in the system. This approach can be considered as adaptive nonlinear dynamic inversion (ANDI). A similar ANDI approach was published in [33].

Later, Lavretsky coins the term CMRAC, which stands for *Combined/Composite Model Reference Adaptive Control*. He suggests using RBFs to perform system identification and estimate the unmatched uncertainties model, with proven stability guarantees, which enables us to apply this method to a larger class of nonlinear systems [39]. Furthermore, Lavretsky uses a Linear Quadratic Regulator (LQR) regulator as a reference model, ensuring optimality of the control law. However, due to the design of an LQR, the states and control input of the system cannot be constrained, meaning if the system has physical constraints they are not accounted for in the control strategy. Similar approaches were proposed in [30, 63, 56].

Finally, Carughi et al. propose to use NNs to reduce model uncertainties in the model and compensate for disturbances online for Model Predictive Controllers (MPC), while guaranteeing proven

stability properties [12]. The advantage of this control strategy is that the control law is optimal, due to MPC, and they increase the predictive capability of the MPC through the NN-observer. Strictly speaking, this approach is not an MRAC since they do not use a model reference. However, this publication is noteworthy, because they were able to provide stability proof for a NN-observer that enhances the performance of the proposed MPC strategy.

## 3.3 Neural Network Controllers

The previous section introduced the concept of CMRACs. They aim to use learning methods to estimate unmatched uncertainties and perform system identification. However, they do not use NNCs, where the output of the NN is the control input to the system. One of the reasons is that standard NNs rely on a gradient descent-based adaptation algorithm. As aforementioned, one of the main instability mechanisms of the gradient descent-based algorithm is due to large adaptation gains [43], which means that the weights of the neural network are potentially unbounded. This is especially the case if the NN is not complex enough to approximate the desired control law and if there are unknown bounded disturbances in the system. Similar gradient descent-based approaches were proposed in [55, 7, 56].

To address these shortcomings, Lewis et al. proposed a direct NN control method for a robotic arm, with guaranteed tracking performance, by imposing a Lyapunov function [41]. An important aspect of their research is that the developed controller does not imitate the behavior of a reference model, but directly learns a control law that converges to a small tracking error, implying no necessity for off-line weight tuning. However, their approach was limited to a maximum of two-layered NNs. Later, Jagannathan et al. proposed the same method for arbitrary-sized feedforward NNCs with basis functions, such as RBF and sigmoid [26]. In both cases, the input to the neural networks are the filtered error dynamics of the system and their approach is limited to fully observable, state-feedback linearizable nonlinear systems.

To address this limitation, Ahmed proposes a structure of NNCs that are provably stable during learning and can also control not state-feedback linearizable nonlinear systems, assuming they can be described in a canonical state variable description form [2]. In this case, the state feedback gain is generated by the output of the neural network and is assumed to be a nonlinear function of the operating point. The proposed approach is valid for SISO and MIMO systems, however it is only defined for RBF NNs. This does not enable the system to introduce constraints, such as saturations and the controller is not guaranteed to be optimal.

The aforementioned approaches use the error as an input to the NNCs. However, in classical control, there are approaches, namely proportional-integral-derivative (PID), that require the error, the history of the error, and the change of the error, as an input. To this end, Thanh et al. introduce a method where the weights of the NNC correspond to the gains of the PID controller [62]. A similar approach was introduced by Cong et al. [13]. Both approaches are compared by Bosshart et al., with the conclusion that even though Thanh et al. do not have stability proof, their approach works better in simulation. This is because the approach of Cong et al. cannot adapt gains individually, cannot adapt to uncertainties, requires a weight initialization, and oscillations may hinder weight adaptation.

The limitation of Thanh et al. is this approach does not have any stability guarantees, because the proportional gain can increase indefinitely [10]. To the best of our knowledge, this control strategy has been applied to a two axes pneumatic artificial muscle manipulator in [62] and in order to compare its performance to other architectures, a second-order system in [10]. Therefore, we take the proposed architecture and apply it to an inverted pendulum.

## 3.4 Noteworthy works on NNC stability

In this thesis, we mainly focus on online learning with stability and model reference control. However, there are a multitude of noteworthy publications on the stability of NNCs, that aim at the a posteriori verification of stability. Hence, in this section, we will briefly introduce a posteriori verification methods.

Typically, in the domain of deep learning, we define a gradient descent-based update law. In terms of control, this comes with the drawback that it becomes nontrivial to determine the convergence properties of the learning algorithm. Usually, we can determine the maximum step size, or learning rate, for the algorithm to converge. However, while solving for the Lipschitz constant, we end up with an equation that is dependent on the initial state and current state of the system. To this end, Fazylab et al. propose a method to find an upper bound on the Lipschitz constant, by solving a semidefinite programming (SDP) problem with integral quadratic constraints for the dynamics [20, 19]. An alternative approach is proposed by Korda in [38, 37], where the NN is re-written as inequality and equality constraints to further constrain the SDP problem. By solving the SDP problem, they find a Lyapunov function that satisfies Lyapunov's stability criteria. However, their approach only takes into account globally stable systems, meaning that locally stable equilibria return the infeasibility of the problem.

Furthermore, there have been attempts to leverage Reinforcement Learning techniques to construct MRAC controllers, where sets of weights can be found using modified versions of the dynamic programming algorithm [21, 45]. In both cases the cost-to-go function of the dynamic programming algorithm is estimated with NNs and the controllers are considered to be NNs as well. Both of these approaches rely on a posteriori stability verification. To this end, they rely on Lyapunov's second method.

## 3.5 Inverted Pendulums

The inverted pendulum serves as an important application for control system design due to its nonlinearity and instability, which makes it difficult to control and difficult to prove that the controlled system is stable [11]. Consequently, there is a multitude of research on the control of inverted pendulum-like systems, using different types of control including, but not limited to the cart inverted pendulum [31, 13, 64, 52, 22, 60], the double pendulum [13, 18, 3, 36], and single arm rotational pendulums [13, 47].

In this thesis we will focus on a single arm pendulum, as shown in Figure 4.1a, called Pendubot. In [9] a self-tuning PID controller is developed for Pendubot. Their approach yields good performance while controlling the system. However, a limitation of this approach is that the control strategy

does not guarantee stabilizing weights in every update step. In contrast, Banerjee et al. develop a Lyapunov-based, output tracking MRAC approach for Pendubot, modelled as a single-input single-output (SISO) system [8]. Similarly, Isdaryani et al. develop an output tracking MRAC architecture for a two-wheeled inverted pendulum, while omitting the translation components [25]. In the end, this boils down to controlling a pendubot-like model, as a simplified two-wheeled inverted pendulum. Ozcelik et al. propose a direct MIMO MRAC strategy for output tracking of an inverted pendulum on a cart, where the states are not fully accessible [50]. In [44] a Lyapunov-based MRAC was investigated for a rotational inverted pendulum, meaning that opposed to having a translational axis, the extra degree of freedom is the yaw angle. Opposed to the aforementioned cases, we investigate a state tracking direct MRAC for Pendubot, since all states are fully accessible for us and we want to use all the available information of our system to control it. Furthermore, we will leverage our insights to construct a nonlinear MRAC update law.

## 3.6 Contribution

In this work, we will take a closer look at stable learning schematics for systems controlled by NNs. More specifically, we commence by developing a direct MRAC for Pendubot, which can be seen as a simplified NNC, that consists of one neuron without an activation function. Furthermore, we develop a PIDNN control strategy for Pendubot, based on the work of [62]. We compare the two strategies to the existing self-tuning PID control strategy from [9].

We then investigate how the concept of stable learning can be extended to design stabilizing update laws for MRACs that use nonlinear NNCs. For simplicity reasons, we start by investigating stable learning for first-order systems. We define an update law based on Lyapunov's second method, meaning that the resulting update laws guarantee the stability of the new set of weights in every time step. In the discussion of this work, we touch upon how this updated schematic could be extended for controlling a MIMO system, such as Pendubot.

This thesis has three main contributions, on the application side, namely 1) the application of the proposed PIDNN by [62] to Pendubot, 2) a state tracking MIMO MRAC strategy for Pendubot, and 3) a comparison of their performances with respect to their ability to control Pendubot and their stability. Additionally, we contribute to the theoretical framework of MRACs, by proposing a stabilizing update law for first-order systems.

# 4 Methodology

This section will explain the chosen methodology of this work. We introduce the used system, the baseline controllers, and the developed controllers throughout this project. These will then be compared, which will be discussed in the results section of this work.

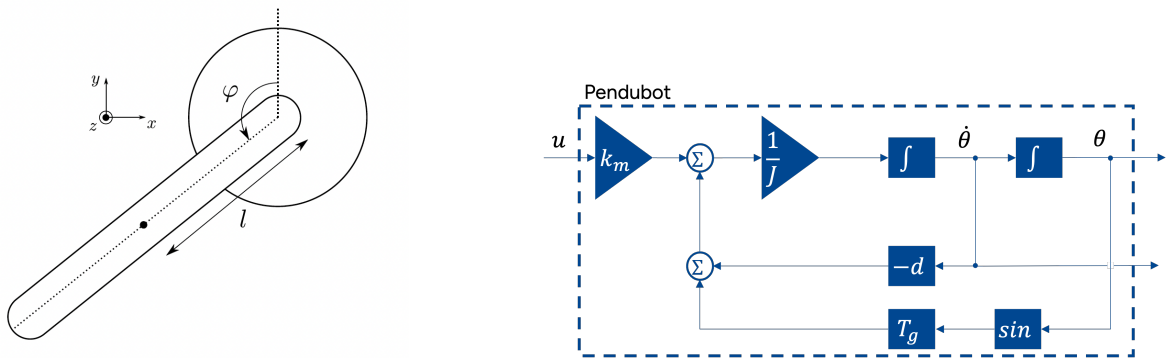
## 4.1 System

Here we will briefly talk about the controlled system, called Pendubot. We explain what the goal of controlling Pendubot is and how to get rid of nonlinearities in the system.

### 4.1.1 Inverted Pendulum, called Pendubot

In this work, we utilize an inverted pendulum, called *Pendubot*, as portrayed in Figure 4.1a, to validate the developed control strategies. It was chosen because it is a simple nonlinear system, with which it is relatively speaking easy to test controllers. The nonlinear dynamics of the 1-DoF inverted pendulum are given by equation (4.1), where the parameters used for simulation are listed in Table 4.1.

$$\ddot{\varphi} = \frac{1}{J}(T_g \sin(\varphi) - d\dot{\varphi} + k_m u) \tag{4.1}$$




(a) Sketch of an inverted pendulum

(b) Blockdiagram inverted pendulum

Figure 4.1: Inveted pendulum, called Pendubot

Pendubot has two equilibria, namely the upright position and the hanging position. With our developed controllers, we aim to stabilize the inverted pendulum in the upright position, which we define as $\varphi = 0$ starting from the hanging position at $\varphi = -\pi$. Note, that the input torque is directly applied at the rotational axis.

### 4.1.2 Nonlinear Dynamic Inversion (NDI)

To get rid of the nonlinearities in the system, we perform nonlinear dynamic inversion (NDI), as introduced in Section 2.2. Specifically, we define $f(\varphi, \dot{\varphi}) = \frac{1}{J}(T_g \sin(\varphi) - d\dot{\varphi})$ and $g(u) = k_m u$. Hence, the resulting control input to the system is now defined by equation (4.2).

| Description | Symbol | Value | Unit |
|---|---|---|---|
| Inertia | $J$ | 0.00565 | $[Nm^2]$ |
| Gravitational Torque | $T_g$ | 0.2747 | $[Nm]$ |
| Damping coefficient | $d$ | 0.005 | $[Nms]$ |
| Motor gain | $k_m$ | 0.05282 | $[Nm/V]$ |

Table 4.1: Numerical values for the inverted pendulum [34]

$$u = \frac{1}{k_m}(J\ddot{\varphi}_d - T_g \sin(\varphi) - d\dot{\varphi}) \tag{4.2}$$

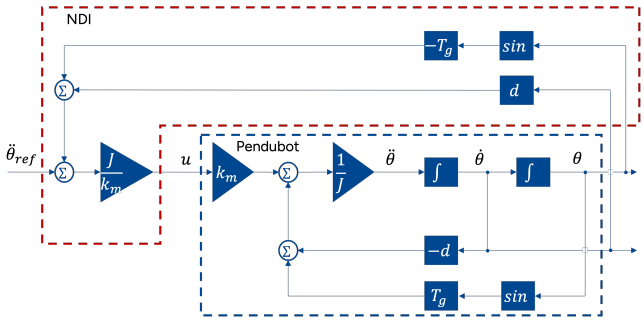



Figure 4.2: Blockdiagram pendubot with nonlinear dynamic inversion

The resulting state space representation of our model is a pure double integrator, as shown in equation (4.3). Hence, after conducting nonlinear dynamic inversion we end up with a linear system, which will simplify the analysis of stability later on in this work.

$$\dot{x} = Ax + Bu = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} x + \begin{bmatrix} 0 \\ 1 \end{bmatrix} u \tag{4.3}$$

From here on, we will refer to the state of the system as $x$, which consists of the angle in the single output case and of the angle and angular acceleration in the multiple output case.

## 4.2 Baseline Controllers

In this work we consider two baseline controllers, namely, 1) an adaptive PID controller, that controls a nonlinear system, and 2) a PIDNN, that introduces another nonlinearity in the controller itself. These will then be compared to a linear MRAC with an LQR as a reference model.

### 4.2.1 Self-tuning PID Controller

We choose to implement or self-tuning PID controller, which we also refer to as an adaptive PID controller, as designed in [9]. This is due to two main arguments, firstly, the self-tuning PID controller can be considered as a linear NNC with a linear activation function, and secondly, assuming we know the system perfectly, it can be easily controlled with a PID controller. Hence, we can use an adaptive strategy to find the stabilizing gains, without having to know the exact dynamics of the

system. The proportional-integral-derivative (PID) controller uses information on the current state error (proportional), past state error (integral), and a future term of the error (derivative), as an input. These are then multiplied with gains for the proportional, integral, and derivative terms respectively. These terms are adaptable according to an update law, based on the error between a model reference and the system output, as shown in Figure 4.3. The update law is a gradient descent-based update law, which makes it difficult to mathematically prove the stability of the resulting controller. Note, that in this notation the state $x$, is the angle of Pendubot.

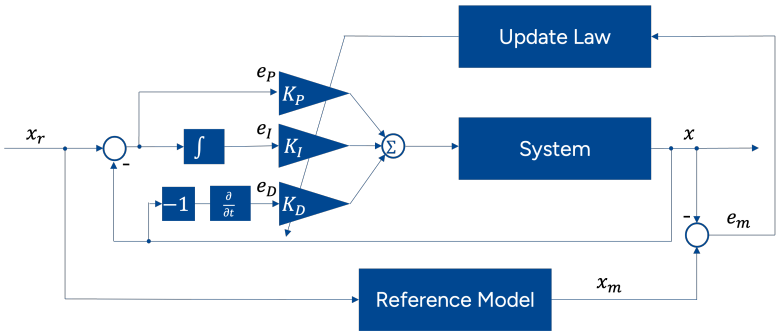



Figure 4.3: Adaptive PID blockdiagram

This proposed approach is single-input-single-output (SISO), which aims to minimize the cost function $J$, as defined in (4.4), where $e_m$ defines the difference between the output of the system and the output of the model reference, and $k$ the time step.

$$J(k) = \frac{1}{2}e_m(k)^2 = \frac{1}{2}(x_m(k) - x(k))^2 \tag{4.4}$$

Using the defined cost function, we update the gains, which can in this case be considered as PID gains according to the following update rule, where $K$ denotes the gains.

$$K(k+1) = K(k) - \eta\frac{\partial J(k)}{\partial K(k)} \tag{4.5}$$

Constructing $\frac{\partial J(k)}{\partial K(k)}$, yields the following expression, with $K = [k_p, k_i, k_d]^T$, and $e = [e_p, e_i, e_d]^T$. The errors are defined as in equation (2.8). Furthermore ,we assume $\frac{\partial x}{\partial u} \approx 1$, as done in [67].

$$\frac{\partial J(k)}{\partial K(k)} = \frac{1}{2}\frac{\partial (x_m(k) - x(k))^2}{\partial x(k)}\frac{\partial x(k)}{\partial u(k)}\frac{\partial u(k)}{\partial K(k)} = -\underbrace{(x_m(k) - x(k))}_{=e_m(k)}e(k) \tag{4.6}$$

Therefore, the update rule for the adaptive PID controller is defined by equation (4.7) for every gain in matrix $K$.

$$\begin{aligned} k_p(k+1) &= k_p(k) + \eta e_m(k)e_p(k) \\ k_i(k+1) &= k_i(k) + \eta e_m(k)e_i(k) \\ k_d(k+1) &= k_d(k) + \eta e_m(k)e_d(k) \end{aligned} \tag{4.7}$$

Additionally, we define an adaptive learning rate. We start with a larger learning rate, which decreases over time. In practice, this means that while approaching the optimal gains, we take smaller

and smaller steps to reduce the chance of overshooting the optimal gains. The adaptive learning rate is defined as in equation (4.8).

$$\eta(k+1) = (1 + \zeta sign(J(k) - J(k-1)))\eta(k) \tag{4.8}$$

### 4.2.2 PIDNN with Activation Function

Additionally, we will consider a PID-Neural Network (PIDNN) controller, first proposed in [62], extended by corrections made by [10]. It also uses a gradient descent-based algorithm to update the weights and incorporates an affine and scaled sigmoid activation function. The general schematic is shown in Figure 4.4.

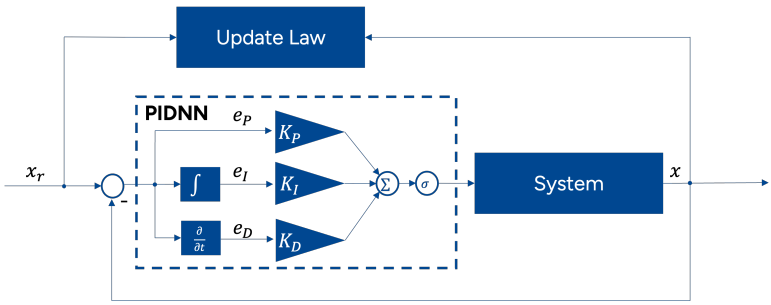



Figure 4.4: PIDNN block diagram, with $\sigma$ being a sigmoid activation function

The learning algorithm minimizes a cost function that uses the difference between the reference signal $x_r$ and the system output $x$. Note, that in this case, we update the weights, i.e. gains of the PID controller according to the difference between the actual output and the reference signal and not according to the difference of a reference model and the output of the system, as opposed to the error concerning a reference model.

For this control strategy, we consider the discrete-time PID errors, as defined in (2.8). Note that this controller applies a sigmoid function to the output of the PID controller, as defined in (2.9). The sigmoid function has the form as defined in (4.9), where the parameter $a$ defines the saturation of the function, i.e. the function will saturate at $\pm\frac{2}{a}$. The resulting gradient is given by equation (4.10). Note we use $\tilde{x}$ in the notation to differentiate between the state vector and the input to the sigmoid function.

$$\sigma(\tilde{x}) = \frac{2(1 - e^{-a\tilde{x}})}{a(1 + e^{-a\tilde{x}})} \tag{4.9}$$

$$\Rightarrow \frac{\partial\sigma(\tilde{x})}{\partial\tilde{x}} = \frac{4e^{-ax}}{(1 + e^{-ax})^2} \tag{4.10}$$

From equation (2.9) and (4.9), the resulting output of the PIDNN is defined as follows:

$$u = \sigma(\tilde{x}) = \sigma(k_p(k)e_p(k) + k_i(k)e_i(k) + k_d(k)e_d(k)) \tag{4.11}$$

Now that the output of the NN is clearly defined, we can construct the update rule. Since we use a

gradient descent-based algorithm, the new set of weights or gains is found by stepping in the opposite direction of the gradient. Hence, it is defined as follows:

$$K(k+1) = K(k) - \eta \frac{\partial J(k)}{\partial K(k)} \tag{4.12}$$

Furthermore, we define the cost function for the gradient descent algorithm as in equation (4.13). Note that as opposed to the self-tuning PID, this update rule does not include a reference model, but only the difference between the desired and real output of the system.

$$J(e_p(k)) = \frac{1}{2} e_p(k)^2 = \frac{1}{2}(x_r(k) - x(k))^2 \tag{4.13}$$

Now let us construct the partial derivative of the cost function concerning the gains of the controller. Again, we assume $\frac{\partial x}{\partial u} \approx 1$, as done in [67]. Here $e(k)$ defines the PID error vector at time step $k$ as defined in (2.8).

$$\frac{\partial J(k)}{\partial K(k)} = \frac{1}{2} \frac{\partial J(k)}{\partial x(k)} \frac{\partial x(k)}{\partial u(k)} \frac{\partial u(k)}{\partial \tilde{x}(k)} \frac{\partial \tilde{x}(k)}{\partial K(k)} = -e_p(k) \frac{4e^{-ax}}{(1+e^{-ax})^2} e(k) \tag{4.14}$$

By inserting equation (4.14) into equation (4.12), we obtain the update law for each gain as defined in equation (4.15).

$$\begin{aligned} k_p(k+1) &= k_p(k) + \eta \frac{4e^{-ax}}{(1+e^{-ax})^2} e_p(k)^2, \\ k_i(k+1) &= k_i(k) + \eta \frac{4e^{-ax}}{(1+e^{-ax})^2} e_p(k) e_i(k) \\ k_d(k+1) &= k_d(k) + \eta \frac{4e^{-ax}}{(1+e^{-ax})^2} e_p(k) e_d(k) \end{aligned} \tag{4.15}$$

## 4.3 Model Reference Adaptive Control

Generally, MRACs have the structure as defined in Figure 4.5. In our case, it has four components, namely 1) the system, 2) the model reference, 3) the adaptive controller, and 4) the adaptation rule, which can be provably stabilizing through leveraging Lyapunov theory. For the design of our MRACs, we use the Pendubot with NDI. The model reference will be an LQR controller for the MIMO case and a first-order system for the SISO case. We use the SISO case to design an update rule, which includes a nonlinearity through an NNC. We begin by deriving the update law for an MRAC for the linear MIMO case, which is stabilizing by design. Based on the insights gained by this implementation, we investigate how we can develop a Lyapunov-based update law for MRACs with nonlinearities.

### 4.3.1 MRACs for State Feedback Control

We will define an MRAC, that learns the control law of a state feedback controller. In this section, we derive the stabilizing update laws based on a combination of MRACs as defined in [39, 6]. Later on, we define the specific feedback controller as a reference model.

We start by deriving the general case MIMO MRAC schematic and this will then be applied to our system at hand. Given an LTI system as in equation (4.16), where $\dot{x} \in \mathbb{R}^n$, $A \in \mathbb{R}^{n \times n}$, $B \in \mathbb{R}^{n \times m}$,

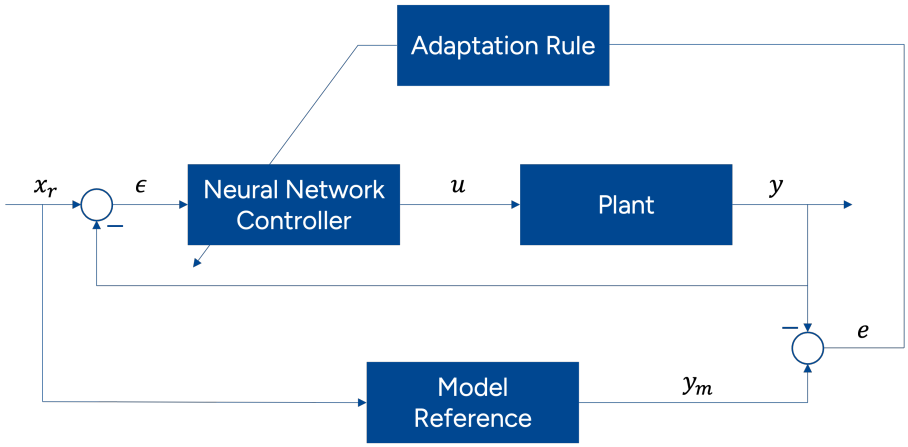



Figure 4.5: MRAC Scheme

$\Lambda \in \mathbb{R}^{m\times m}$, $x \in \mathbb{R}^n$, and $u \in \mathbb{R}^m$. Note that in this case, $A$, $\Lambda$ are unknown.

$$\dot{x} = Ax + B\Lambda u \tag{4.16}$$

Furthermore, we define a reference model, which will dictate the desired behavior, where $\dot{x}_m \in \mathbb{R}^n$, $A_m \in \mathbb{R}^{n\times n}$, $B_m \in \mathbb{R}^{n\times n}$, $x \in \mathbb{R}^n$, and $x_r \in \mathbb{R}^n$.

$$\dot{x}_m = A_m x_m + B_m x_r \tag{4.17}$$

If we knew the dynamics of the system, we could design the ideal controller. Hence, we define the ideal control input $u_{ideal} = k_x^T x + k_r^T r$ and get the ideal dynamics of the closed-loop system in equation (4.18).

$$\dot{x} = Ax + B\Lambda(k_x^T x + k_r^T x_r) = (A + B\Lambda k_x^T)x + B\Lambda k_r^T x_r \tag{4.18}$$

By comparing the ideal dynamics to the model reference, we can now define the matching conditions in equation (4.19), which we will need later on.

$$\text{(I)} \quad A_m = A + B\Lambda k_x^T \qquad \text{(II)} \quad B_m = B\Lambda k_r^T \tag{4.19}$$

Now we can define the control input to the real system, in the same form of the ideal input $u = \hat{k}_x^T x + \hat{k}_r^T x_r$, where $\hat{k}_x \in \mathbb{R}^{n\times m}$, and $\hat{k}_r \in \mathbb{R}^{n\times m}$. By plugging the control input into equation (4.16), we get equation (4.20).

$$\dot{x} = Ax + B\Lambda(\hat{k}_x^T x + \hat{k}_r^T x_r) = (A + B\Lambda \hat{k}_x^T)x + B\Lambda \hat{k}_r^T x_r \tag{4.20}$$

Since the update of the weights is dependent on the error between the reference model and the output of the system, we define the error next as defined in equation (4.21) and its time derivative in equation (4.22).

$$e = x - x_m \tag{4.21}$$

$$\begin{aligned}\dot{e} = \dot{x} - \dot{x}_m &= (A + B\Lambda\hat{k}_x^T)x + B\Lambda\hat{k}_r^T x_r - A_m x_m + B_m x_r \pm A_m x \\ &= A_m \underbrace{(x - x_m)}_{=e} + (A + B\Lambda\hat{k}_x^T - A_m)x - (B\Lambda\hat{k}_r^T - B_m)x_r\end{aligned} \tag{4.22}$$

By substituting the matching conditions from equation (4.19), we end up with equation (4.23).

$$\begin{aligned}\Rightarrow \dot{e} &= A_m e + (A + B\Lambda\hat{k}_x^T - A - B\Lambda k_x^T)x + (B\Lambda\hat{k}_r^T - B\Lambda k_r^T)x_r \\ &= A_m e + B\Lambda(\underbrace{\hat{k}_x - k_x}_{=\Delta K_x})^T x + B\Lambda(\underbrace{\hat{k}_r - k_r}_{=\Delta K_r})^T x_r\end{aligned} \tag{4.23}$$

In the next step, we choose a Lyapunov candidate and construct an update law, in a way that the time derivative of the Lyapunov function is negative for all time steps. The Lyapunov candidate is defined as equation (4.24), where $\mathrm{tr}(\cdot)$ is the trace function, $P \in \mathbb{R}^{n\times n}$ is a positive definite matrix, and $\Gamma_x \in \mathbb{R}^{n\times n}$, $\Gamma_r \in \mathbb{R}^{n\times n}$ are adaptive rates or learning rates, where $\Gamma_x$, $\Gamma_r$ are symmetric positive definite matrices.

$$V(e, \Delta K_x, \Delta K_r) = e^T P e + \mathrm{tr}(\left[\Delta K_x^T \Gamma_x^{-1} \Delta K_x + \Delta K_r^T \Gamma_r^{-1} \Delta K_r\right] \Lambda) \tag{4.24}$$

To ensure the decreasing energy of the system as $t \to \infty$, we take the time derivative of the Lyapunov candidate, resulting in equation (4.25), where $Q$ is positive definite and satisfies the Lyapunov equation, as defined in (2.7).

$$\begin{aligned}\Rightarrow \dot{V}(e, \Delta K_x, \Delta K_r) =& \dot{e}^T P e + e^T P \dot{e} + 2\mathrm{tr}(\left[\Delta \dot{K_x}^T \Gamma_x^{-1} \Delta K_x + \Delta \dot{K_r}^T \Gamma_r^{-1} \Delta K_r\right] \Lambda) \\ =& (A_m e + B\Lambda\Delta K_x^T x + B\Lambda\Delta K_r^T x_r)^T P e + e^T P (A_m e + B\Lambda\Delta K_x^T x + B\Lambda\Delta K_r^T x_r) \\ &+ 2\mathrm{tr}(\left[\Delta \dot{K_x}^T \Gamma_x^{-1} \Delta K_x + \Delta \dot{K_r}^T \Gamma_r^{-1} \Delta K_r\right] \Lambda) \\ =& e^T \underbrace{(A_m^T P + P A_m)}_{=-Q} e + 2e^T P (B\Lambda\Delta K_x^T x + B\Lambda\Delta K_r^T x_r) \\ &+ 2\mathrm{tr}(\left[\Delta \dot{K_x}^T \Gamma_x^{-1} \Delta K_x + \Delta \dot{K_r}^T \Gamma_r^{-1} \Delta K_r\right] \Lambda) \\ =& - e^T Q e + 2(\underbrace{e^T P B\Lambda\Delta K_x^T x}_{\text{(III)}} + \underbrace{e^T P B\Lambda\Delta K_r^T x_r}_{\text{(IV)}}) \\ &+ 2\mathrm{tr}(\left[\Delta \dot{K_x}^T \Gamma_x^{-1} \Delta K_x\right] \Lambda) + 2\mathrm{tr}(\left[\Delta \dot{K_r}^T \Gamma_r^{-1} \Delta K_r\right] \Lambda)\end{aligned} \tag{4.25}$$

Now, using the vector trace identity, meaning $\mathrm{tr}(b^T a) = a^T b$, for $a, b \in \mathbb{R}^n$, we can reformulate (III) and (IV) to (IIIa) and (IVa) and thus equation (4.25) to equation (4.27).

$$\text{(IIIa)} \quad \underbrace{e^T P B\Lambda}_{a^T} \underbrace{\Delta K_x^T x}_{b} = \mathrm{tr}(\underbrace{\Delta K_x^T x}_{b} \underbrace{e^T P B\Lambda}_{a^T}) \qquad \text{(IVa)} \quad \underbrace{e^T P B\Lambda}_{a^T} \underbrace{\Delta K_r^T r}_{b} = \mathrm{tr}(\underbrace{\Delta K_r^T r}_{b} \underbrace{e^T P B\Lambda}_{a^T}) \tag{4.26}$$

$$
\begin{aligned}
\Rightarrow \dot{V}(e, \Delta K_x, \Delta K_r) = &- e^T Q e \\
&+ 2\mathrm{tr}(\Delta K_x^T \underbrace{(\Gamma_x^{-1} \Delta \dot{K}_x + x e^T P B)}_{\text{(V)}} \Lambda) \\
&+ 2\mathrm{tr}(\Delta K_r^T \underbrace{(\Gamma_r^{-1} \Delta \dot{K}_r + r e^T P B)}_{\text{(VI)}} \Lambda)
\end{aligned}
\tag{4.27}
$$

Since $Q$ is positive definite, we know that $-e^T Q e \leq 0$. To guarantee that $\dot{V}(e, \Delta K_x, \Delta K_r) \leq 0$, we need to ensure the two remaining terms of equation (4.27) are less or equal to zero. One way is to take parts (V) and (VI) and make sure that they are equal to zero and if we reorder these to terms, concerning $\Delta \dot{K}_x$ and $\Delta \dot{K}_r$, we have an update rule for our time-dependent gains. The resulting update rules are defined in equation (4.28).

$$
\Delta \dot{K}_x = -\Gamma_x x e^T P B \qquad \Delta \dot{K}_r = -\Gamma_r r e^T P B \tag{4.28}
$$

Now let us re-iterate the key points of the construction of the update laws. By far the most important assumption is that a set of optimal gains exist, even though we do not know them. These are called the matching conditions, and by substituting them into equation (4.22) we can get rid of the internal dynamics of the system $A$ in the time derivative of the error. This in turn ensures that the derivative of the Lyapunov candidate is not dependent on $A$. Hence, we can construct update laws, that are independent of the internal dynamics of the system. The only knowledge we need of the system is how an input to the system influences the state of the system, meaning the matrix $B$. Furthermore, through being able to cancel out the $A$ matrix in the Lyapunov candidate, we are now able to factor out the term of the optimal gains. Due to this, we do not need to worry about knowing them since they are not needed to nullify the terms (V) and (VI) in equation (4.27).

Another aspect we want to point out is that all components of the MRAC are linear transformations. The only time-varying component are the gains that are being updated over time. Thus, the MRAC structure can also be considered as *real time* pole-placement, while guaranteeing stable poles in every update step.

### 4.3.2 Nonlinear MRACs for First-Order Systems

One of the goal of this work is to analyse stability properties of nonlinear NNCs. To this end, we propose stable update scheme. We commence by looking at the simpler case of a first-order system. More, specifically, we define a problem, where we show that the even in the nonlinear case, we are able to learn a parameter with an algorithm that is based on the logic of a linear MRAC. We call this nonlinear Model Reference Adaptive Control (NMRAC). The general schematic can be seen in Figure 4.6.

We desire to control a system of the form, as described in equation (4.29), where $\phi : \mathbb{R} \to \mathbb{R}$ is a nonlinear function, $e_x = x_r - x$,and $\theta$ is the parameter we aim to learn. Here, we would like to point that $\theta$ can be seen as a weight, and $\phi$ the activation function of a NN of the form of a sigmoidial saturation function, as described in equation (4.9).

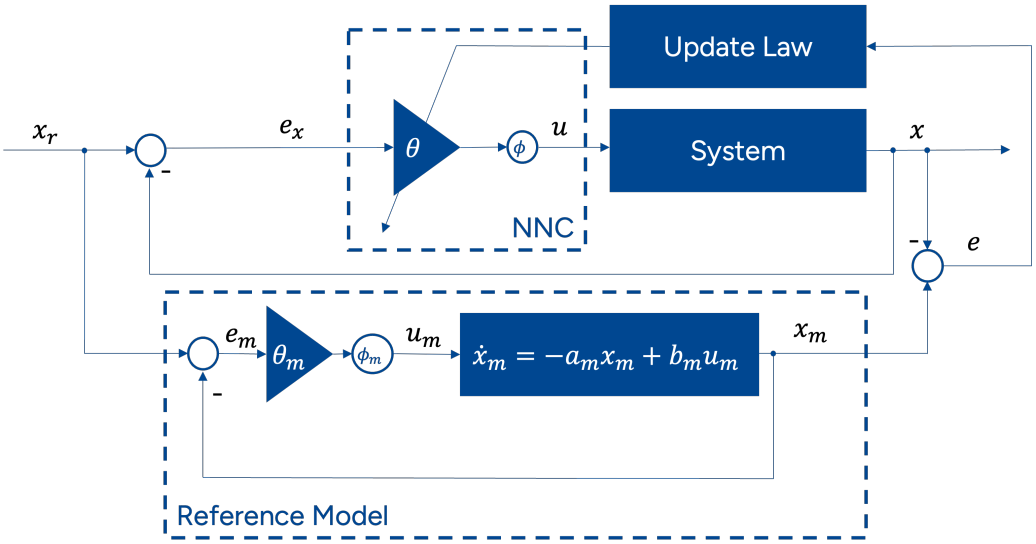



Figure 4.6: MRAC for first-oder systems controlled by simple NNCs

$$\dot{x} = -ax + b\phi(\theta e_x) \tag{4.29}$$

Next, we define a stable reference model, as defined in equation (4.30), where $e_m = x_r - x_m$, and $a_m > 0$. Note, that we want $\phi_m$ to be of the same form as $\phi$.

$$\dot{x}_m = -a_m x_m + b_m \phi_m(\theta_m e_m) \tag{4.30}$$

Since we aim for the system to follow the reference model, we define the error dynamics as in equation (4.31). Note, that from this definition it follows that $e = e_x - e_m$. We will use this alternative definition later for the stability analysis of the update law.

$$e = x_m - x \tag{4.31}$$

Following the error dynamics, we define the Lyapunov candidate to be defined as in equation (4.32). The factor $c > 0$, is used to accelerate or decelerate the learning process. The norm of the error captures the distance between the internal states of the reference model and the system, and the term $||\alpha||_2^2$ captures the distance between the NNC and the desired NNC, which can be seen as the difference in dynamics between the controlled system and the model reference. Note, that both $e$ and $\alpha$ should be 0, when our system follows the reference model and the desired parameters are learned. Futhermore, this property renders our Lyapunov candidate to be positive definite.

$$V(e, \alpha) = ||e||_2^2 + ||c\alpha||_2^2 \tag{4.32}$$

To ensure that the controlled system is stable, we require the time derivative of the Lyapunov function to be negative. Therefore, we analyze the behavior of the resulting time derivative of the Lyapunov function. The resulting equation is defined in (4.33).

$$\dot{V}(e,\alpha) = 2e\dot{e} + 2\alpha\dot{\alpha}c \tag{4.33}$$

We now construct the time derivative of the error dynamics, which is defined by equation (4.34). We extend the equation by $\pm(a_m x b_m \phi_m(\theta_m e_x))$, to construct the term $\alpha$, which is now dependent on $e_x$, and $\theta$. We are then left with two other terms, namely, $-a_m e$, and $\gamma_m(e_m, e_x)$.

$$\begin{aligned}\dot{e} =& \dot{x}_m - \dot{x} = -a_m x_m + b_m\phi_m(\theta_m e_m) - (-ax + b\phi(\theta e_x)) \pm (a_m x + b_m\phi_m(\theta_m e_x)) \\ =& -a_m \underbrace{(x_m - x)}_{=e} + \underbrace{(a - a_m)x + b_m\phi_m(\theta_m e_x) - b\phi(\theta e_x)}_{=\alpha(e_x,\theta)} + \underbrace{b_m(\phi_m(\theta_m e_m) - \phi_m(\theta_m e_x))}_{=\gamma_m(e_m,e_x)}\end{aligned} \tag{4.34}$$

$$\begin{aligned}\dot{V}(e,\alpha) =& 2e(-a_m e + \alpha(e_x,\theta) + \gamma_m(e_m,e_x)) + 2\alpha c\dot{\alpha}(e_x,\theta) \\ =& -2a_m e^2 + 2e\gamma_m(e_m,e_x) + 2\alpha(e_x,\theta)(e + c\dot{\alpha}(e_x,\theta))\end{aligned} \tag{4.35}$$

By substituting equation (4.34) into equation (4.33), we get equation (4.35).

Note, that the first term $-2a_m e^2$ is always negative, since $a_m > 0$, and $e^2 > 0$. The second term will always be negative, due to the property of $e = e_x - e_m$. It follows that if $e < 0 \Rightarrow \gamma_m(e_m, e_x) > 0$, which implies that the second term is negative, and if $e > 0 \Rightarrow \gamma_m(e_m, e_x) < 0$, which again implies that the second term is negative. Hence, the second term reamins negative.

Since, we can already ensure that the first two terms of equation (4.35) are negative, it remains to show that the last term is always negative or equal to zero at all times. Since $\theta$ is dependent on time, this implies $\dot{\alpha}(e_x,\theta)$ will have a term that includes $\dot{\theta}$. Therefore, we choose $\dot{\theta}$, such that the third term in equation (4.35) is nullified. Note, that due to the way we construct the update law, stability is guaranteed, since the Lyapunov function is positive definite and simultaneously the decrease condition is satisfied.

$$\begin{aligned}0 =& 2\alpha(e_x,\theta)(e + \dot{\alpha}(e_x,\theta)) \\ \Leftrightarrow 0 =& e + c\dot{\alpha}(e_x,\theta) \\ \Leftrightarrow 0 =& e + c\left[(a - a_m)\dot{x} + b_m\frac{d\phi_m(\theta_m e_x)}{dt} - b\frac{d\phi(\theta e_x)}{dt}\right] \\ \Leftrightarrow 0 =& \frac{1}{c}e + (a - a_m)\dot{x} + b_m\frac{\partial\phi_m(\tilde{x})}{\partial\tilde{x}}|_{\tilde{\theta}e_x=\theta e_x}\theta_m\dot{e}_x - b\frac{d\phi(\tilde{x})}{\partial\tilde{x}}|_{\tilde{x}=\theta e_x}(e_x\dot{\theta} + \theta\dot{e}_x)\end{aligned} \tag{4.36}$$

$$\Rightarrow \dot{\theta} = \frac{1}{e_x b\frac{\partial\phi(\tilde{x})}{\partial\tilde{x}}|_{\tilde{x}=\theta e_x}}\left(\frac{1}{c}e + (a - a_m)\dot{x} + b_m\frac{\partial\phi_m(\tilde{x})}{\partial\tilde{x}}|_{\tilde{x}=\theta_m e_x}\theta_m\dot{e}_x\right) - \frac{\theta}{e_x}\dot{e}_x \tag{4.37}$$

When taking a closer look at the update rule in equation (4.37), it follows that we require to implement an update hold, when $e_x = 0$. This is reasonable, since whenever the state of our reference system is equal to the reference signal, we will not be able to extract information on how to change the weights in order for our system to converge towards the reference signal. In practice this means that we implement a threshold $\varepsilon$, where we do not update the weights.

### 4.3.3 Nonlinear MRACs for Multiple-Input, Multiple-Output Systems

In this work we additionally propose an update law for multiple-input, multiple-output (MIMO) nonlinear MRACs. This section is dedicated to proposing the MIMO update law, based on the same logic as discussed in the previous sections. The update law will then be simulated använder the results are presented in section 5.3.4. The MIMO case is more interesting than the first-order case since the state variables can influence each other and driving the system to a desired state is more challenging. In this part of the discussion, we will share our insights on constructing a MIMO update law, based on the proposed diagram.

The system is defined as in equation (4.38). Furthermore, we define a reference model that imposes a desired behavior on the system, as defined in equation (4.39). Note, that we assume $A, A_m \in \mathbb{R}^{n\times n}$, $B, B_m \in \mathbb{R}^{n\times m}$, $\theta, \theta_m \in \mathbb{R}^{m\times n}$, and $\phi, \phi_m$ are applied element-wise.

$$\dot{x} = Ax + B\phi(\theta e_x), \quad e_x = r - x \tag{4.38}$$

$$\dot{x}_m = A_m x_m + B_m \phi_m(\theta_m e_m) \quad e_m = r - x_m \tag{4.39}$$

We define the internal error of the system with respect to the model reference $e = x_m - x$, which results in the time derivative of the error to be as defined in equation (4.40).

$$\begin{aligned}
\dot{e} &= \dot{x}_m - \dot{x} \\
&= A_m x_m + B_m\phi_m(\theta_m e_m) - [Ax + B\phi(\theta e_x)] \pm [A_m x + B_m \phi_m(\theta_m e_x)] \\
&= A_m e + \underbrace{B_m\phi_m(\theta_m e_m) - B_m\phi_m(\theta_m e_x)}_{=\gamma_m(e_m, e_x)} + \underbrace{(A_m - A)x + B_m\phi_m(\theta_m e_x) - B\phi(\theta e_x)}_{:=\alpha(e_x,\theta)} \\
&= A_m e + \gamma_m(e_m, e_x) + \alpha(ex,\theta)
\end{aligned} \tag{4.40}$$

Note, that the term $\alpha$ describes the difference in the dynamics.

We define the following Lyapunov candidate, which is a multidimensional version of the special case considered in (4.32).

$$V(e,\theta) = ||e||_P^2 + ||\alpha(e_x,\theta)||_{\Gamma^{-1}}^2 = e^T P e + \alpha(e_x,\theta)^T \Gamma^{-1} \alpha(e_x,\theta) \tag{4.41}$$

Here $P, \Gamma^{-1} \in \mathbb{R}^{n\times n}$ are positive definite matrices. Note, that the Lyapunov function consists of the squared norm of the error $e$, and the squared norm of the difference in dynamics $\alpha(e_x,\theta)$, rendering the Lyapunov function to be positive definite. Hence, this function can only be equal to zero, when the internal error and the differnce in dynamics are exactly equal. Additionally, showing the negative definitness of its time derivative results in stability of the closed-loop system during the learning process.

Its time derivative is given by (4.42).

$$
\begin{aligned}
\dot{V}(e,\theta) &= \dot{e}^T P e + e^T P \dot{e} + \dot{\alpha}(e_x,\theta)^T \Gamma^{-1} \alpha(e_x,\theta) + \alpha(e_x,\theta)^T \Gamma^{-1} \dot{\alpha}(e_x,\theta) \\
&= (A_m e + \gamma_m(e_m,e_x) + \alpha(e_x,\theta))^T P e + e^T P (A_m e + \gamma_m(e_m,e_x) + \alpha(e_x,\theta)) \\
&+ 2\alpha(e_x,\theta)^T \Gamma^{-1} \dot{\alpha}(e_x,\theta) \\
&= e^T (A_m^T P + P A_m) e^T + 2 e^T P \gamma(e_m,e_x) + 2 e^T P \alpha(e_x,\theta) + 2\alpha(e_x,\theta)^T \Gamma^{-1} \dot{\alpha}(e_x,\theta) \\
&= \underbrace{-e^T (A_m^T P + P A_m) e^T}_{\text{(I)}} + \underbrace{2 e^T P \gamma(e_m,e_x)}_{\text{(II)}} + \underbrace{2\alpha(e_x,\theta)^T (P e + \Gamma^{-1} \dot{\alpha}(e_x,\theta))}_{\text{(III)}}
\end{aligned}
\tag{4.42}
$$

Following the logic of linear MRAC, term $(III)$ is used to define the update law through nullification. Hence, it remains to show that the sum of terms $(I)$ and $(II)$ are negative. The negative definiteness of these two terms are system dependent. In section 5.3.4, we will show that there exists at least one system, where the sum of terms remains negative. To continue with the derivation of the update law, we will for now assume that the sum of these two terms remain negative.

To derive the MIMO update law, we require term $(III)$ in (4.42) to be equal to zero. The resulting equation is shown in (4.43).

$$
\begin{aligned}
Pe &= -\Gamma^{-1} \dot{\alpha}(e_x,\theta) \\
Pe &= -\Gamma^{-1} \left( (A_m - A)\dot{x} + B_m \frac{d\phi_m(\theta_m e_x)}{dt} - B \frac{d\phi(\theta e_x)}{dt} \right) \\
Pe &= -\Gamma^{-1} \left( (A_m - A)\dot{x} + B_m \frac{\partial \phi_m(\tilde{x})}{\partial \tilde{x}}|_{\tilde{x}=\theta_m e_x} \theta_m \dot{e}_x - B \frac{\partial \phi(\tilde{x})}{\partial \tilde{x}}|_{\tilde{x}=\theta e_x} (\dot{\theta} e_x + \theta \dot{e}_x) \right) \\
B \frac{\partial \phi(\tilde{x})}{\partial \tilde{x}}|_{\tilde{x}=\theta e_x} \dot{\theta} e_x &= \Gamma P e + (A_m - A)\dot{x} + B_m \frac{\partial \phi_m(\tilde{x})}{\partial \tilde{x}}|_{\tilde{x}=\theta_m e_x} \theta_m \dot{e}_x - B \frac{\partial \phi(\tilde{x})}{\partial \tilde{x}}|_{\tilde{x}=\theta e_x} \theta \dot{e}_x
\end{aligned}
\tag{4.43}
$$

Since, $B \frac{\partial \phi(\tilde{x})}{\partial \tilde{x}}|_{\tilde{x}=\theta e_x} \in \mathbb{R}^{n \times m}$, the matrix is not invertible. Therefore, we take the Moore-Penrose inverse of the matrix, which we define as in equation (4.44), thus yielding (4.45).

$$
\Sigma^\dagger := ((B \frac{\partial \phi(\tilde{x})}{\partial \tilde{x}}|_{\tilde{x}=\theta e_x})^T B \frac{\partial \phi(\tilde{x})}{\partial \tilde{x}}|_{\tilde{x}=\theta e_x})^{-1} (B \frac{\partial \phi(\tilde{x})}{\partial \tilde{x}}|_{\tilde{x}=\theta e_x})^T \tag{4.44}
$$

$$
\Rightarrow \dot{\theta} e_x = \Sigma^\dagger \underbrace{(\Gamma P e + (A_m - A)\dot{x} + B_m \frac{\partial \phi_m(\tilde{x})}{\partial \tilde{x}}|_{\tilde{x}=\theta_m e_x} \theta_m \dot{e}_x - B \frac{\partial \phi(\tilde{x})}{\partial \tilde{x}}|_{\tilde{x}=\theta e_x} \theta \dot{e}_x)}_{:=\xi} \tag{4.45}
$$

Recall, $\dot{\theta} \in \mathbb{R}^{m \times n}$, $e_x \in \mathbb{R}^n$, $\Sigma^\dagger \in \mathbb{R}^{m \times n}$, $\xi \in \mathbb{R}^n$, and we desire to solve for $\dot{\theta}$. The solution of this equation is given by equation (4.46), which corresponds to the MIMO nonlinear MRAC update law. Note, $e_x^\dagger$ is the pseudo-inverse of $e_x$, which is defined as $e_x^\dagger = 0$, if $e_x = 0$, and $e_x^\dagger = e_x^T (e_x^T e_x)^{-1}$ otherwise.

$$
\dot{\theta} = \Sigma^\dagger \xi e_x^\dagger \tag{4.46}
$$

This system of equations always has an infinite number of solutions, except if $e_x = 0$, and simultaneously $\Sigma^\dagger \xi \neq 0$. Whenever, this case occurs, we propose to not update the weights of the neural

network. However, when there is an infinite number of solutions, we can choose values for $\dot{\theta}$, that satisfy this equation. For example, we could choose the solution that minimizes the spectral norm of $\dot{\theta}$. The idea is based on [68, 28], where it was proposed to minimize the spectral norm of the Jacobian of the NN, to improve its robustness properties. In our case, this would correspond to taking the solution, where $\dot{\theta}$ has the smallest maximum singular value. By minimizing the norm, we ensure that no direction in the weight space undergoes excessively large changes during training. This can help in preventing sudden changes of $\theta$, and aid towards a more smooth learning process.

# 5 Results

In this chapter, we will present our simulation results. The simulations are done in MATLAB/Simulink for continuous control strategies and in Python 3.11 for discrete-time control strategies. The implementations in Python, mainly use Numpy for mathematical operations. We commence with implementing the baseline models, namely the self-tuning PID controller and the PIDNN. Subsequently, we will talk about our implementation of a Model Reference Adaptive Controller (MRAC). A summary of the simulation results for Pendubot is given in Table 5.1.

| Model | Overshoot | Settling Time | Guaranteed Stability |
|---|---|---|---|
| Self-tuning PID | $> 100\%$ | 5 sec | $\times$ |
| PIDNN | $\pm 50\%$ | 10 sec | $\times$ |
| Linear MRAC | - | 7.5 sec | ✓ |

Table 5.1: Performance comparison for controllers for Pendubot

## 5.1 Baseline Controllers

In this section we present the simulation results of the self-tuning PID and the PIDNN.

### 5.1.1 Self-Tuning PID

The self-tuning PID controller was simulated for 100 seconds, with a sampling time of 0.0001. We set the learning rate to be $\eta = 0.001$. Furthermore, we set the initial weights to be $K_P = 0$, $K_I = 0$, and $K_D = 0$. Additionally, we use an adaptive learning rate as defined in Section 4.2.1. The simulation results can be seen in Figure 5.1. We can observe that in the first 10 seconds of the simulation, the system overshoots the reference signal by more than 100%, which can be interpreted as a full turn. However, we can observe that with time, specifically after less than 5 seconds, the system behavior converges towards the behavior of the reference model.

Furthermore, if we take a look at the evolution of gains over time, as shown in Figure 5.2, we can see that the proportional part takes a relatively high value, compared to the other two gains and converges to a value that is slightly above 80. Opposed to the proportional gain, the integral gain settles at 0, and the derivate term settles at a value that is slightly lower than 10. These weights imply that to control the system, we do not need a full PID controller, but a PD controller is enough for the system to exhibit the desired behavior. Note, that the PD controller to be equal to a state feedback controller, where the derivative term is an estimate of the angular velocity.

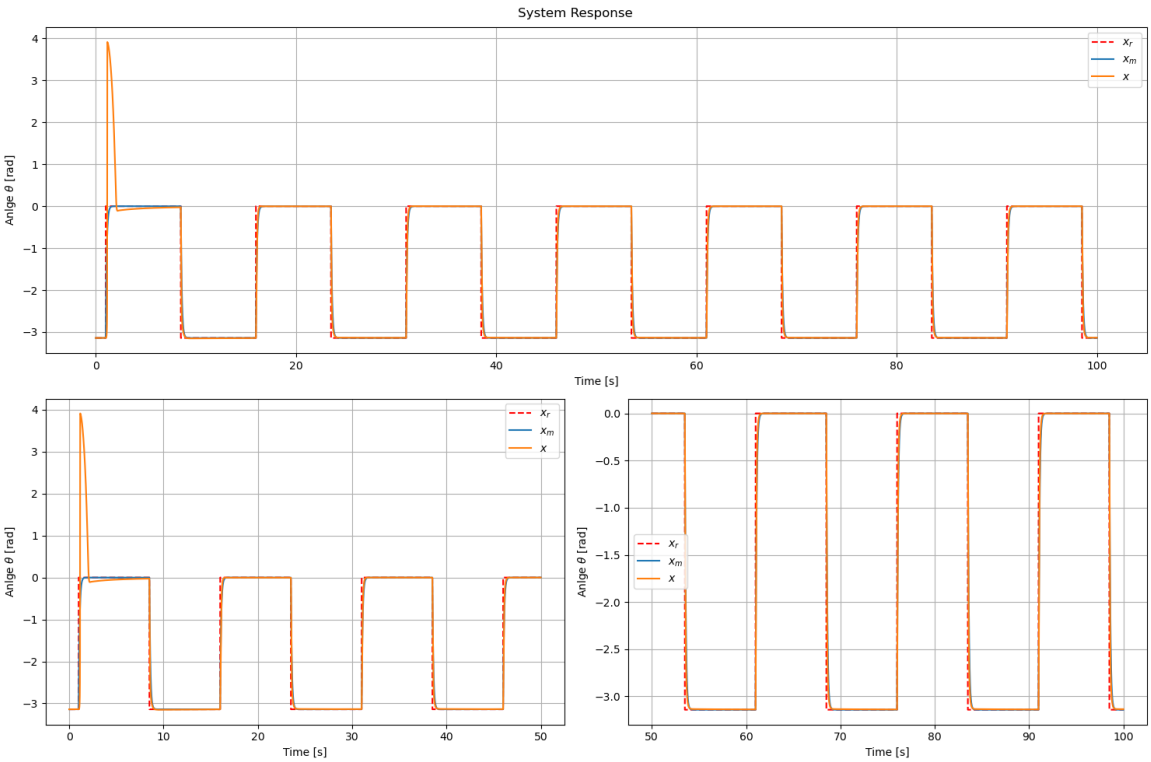



Figure 5.1: Self-tuning PID with linear activation function

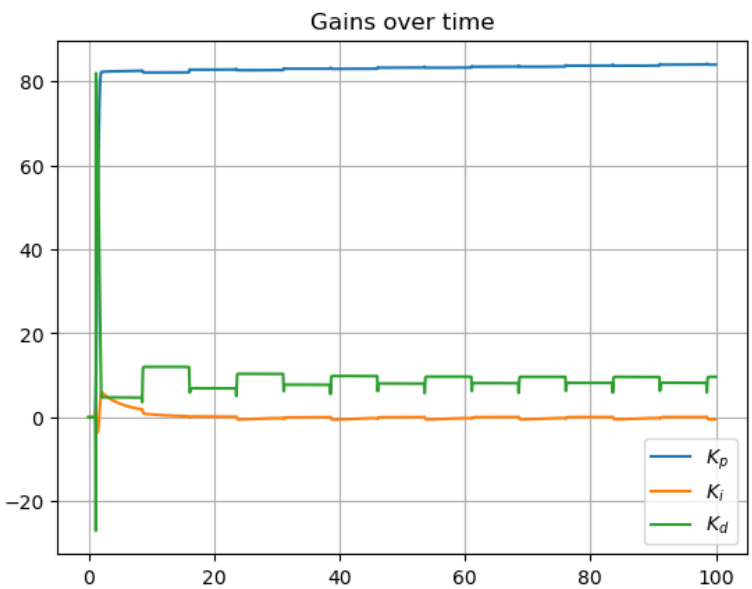



Figure 5.2: Self-tuning PID gains over time

### 5.1.2 PID Neural Networks for Pendubot

The implementation of PIDNNs follows the SISO implementation proposed by Thanh et al. [62]. Their implementation is used to control a 2 axes pneumatic artificial muscle manipulator using a neural network. Then Bosshart et al. use the same approach, adjusted by a few corrections, and apply it to a second-order system [10]. In this work, we apply the proposed control strategy to Pendubot.

In this case, we perform NDI, as described in 2.2. Hence, the resulting plant is a pure double integrator. We choose a simulation horizon of 50 seconds, with a sampling time of $T_s = 0.0001$. Furthermore, we choose an adaptation rate of $\eta = 0.001$, a saturation parameter of $a = 0.1$. Note, that we use a smoothed pulse function as a reference signal, which is constructed using linear combinations of sigmoidial functions. The initial weights of the NN are given by $k_p = k_i = k_d = 0$.

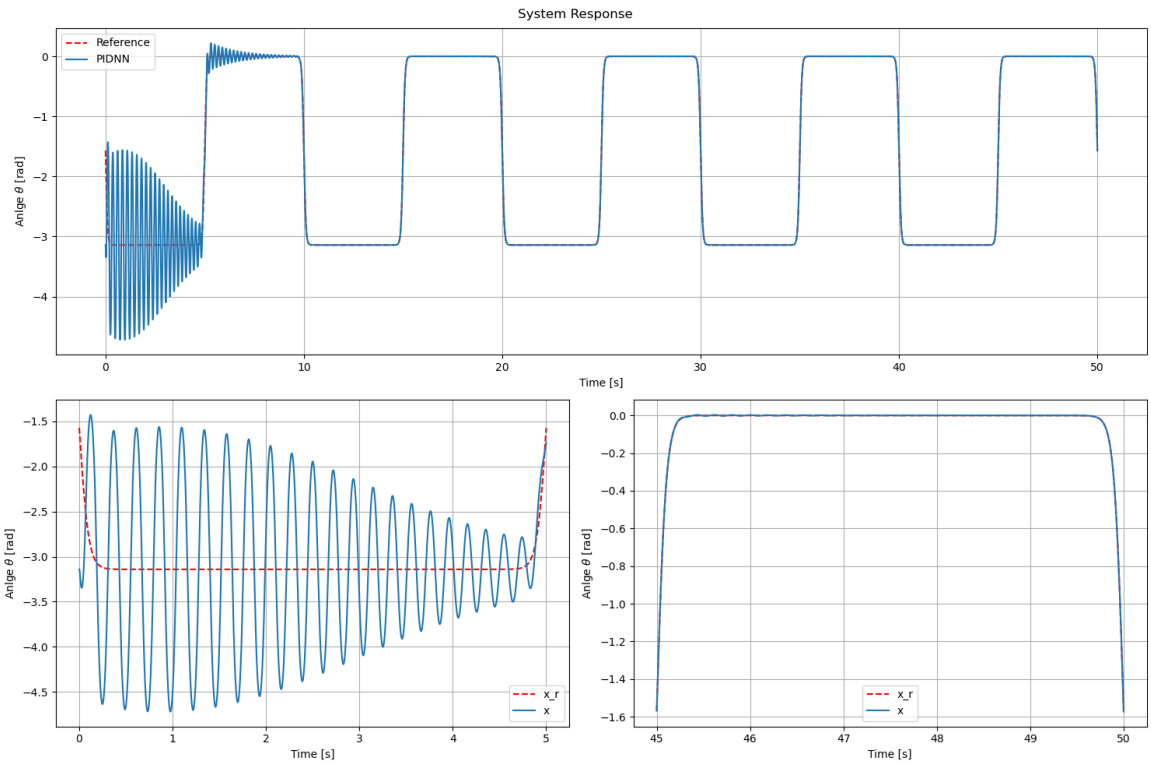



Figure 5.3: Pendubot response with PIDNN

The response of the controlled system is shown in Figure 5.3. At the beginning of the simulation horizon, the system oscillates around the desired value. The oscillation is around 50% higher and lower than the desired response. However, the oscillation decreases over time and after a bit less than 10 seconds the system found a set of weights that make the controlled system exactly follow the desired output.

The evolution of the PID gains is shown in Figure 5.4. We can observe that the proportional gain increases quickly and settles at a value slightly above 30. Both the integral and the derivative terms oscillate at the beginning of the simulation horizon and after a bit less than 10 seconds, the values converge to 0 for the integral gain and 14 for the derivate gain. All three gains reach a stationary value

and converge after about 10 seconds. This can be explained by the observation that the controlled system exactly follows the reference signal after 10 seconds. The update law for all three gains is multiplied by the difference between the reference signal and the system output. When the system exactly follows the reference signal, this term is equal to 0, thus resulting in the gains not being updated.

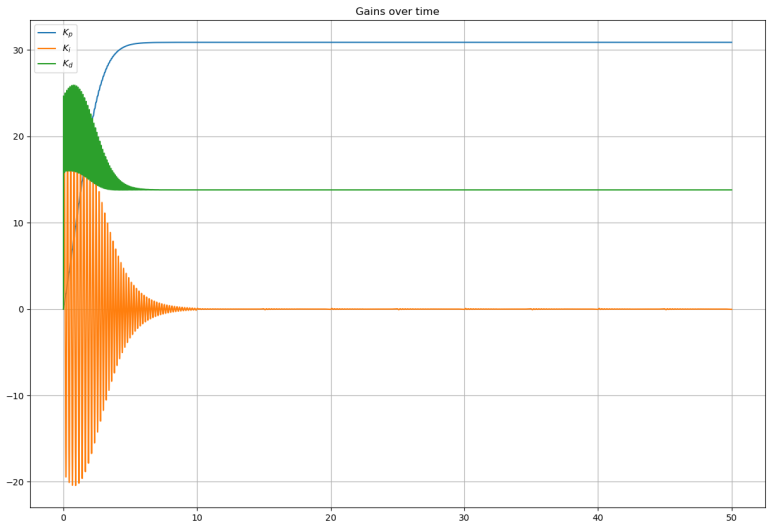



Figure 5.4: Gains PIDNN for Pendubot

## 5.2 MRAC for Linear State Feedback Control

For our simulation of MRAC, we aim to learn stable weights for a linear state feedback controller. The reference model is chosen to be a second-order system that differs from the system. Its dynamics are defined as in equation (5.1), and it is controlled by an LQR controller, as indicated in Figure 5.5. It is constructed according to Section 2.4.3 for the system as described in (4.3). The identified optimal feedback gains for the LQR are $L_1 = 1$, and $L_2 = 3$.

$$A_{ref} = \begin{pmatrix} 0 & 1 \\ 0 & 1 \end{pmatrix} \qquad B_{ref} = \begin{pmatrix} 0 \\ 1 \end{pmatrix} \tag{5.1}$$

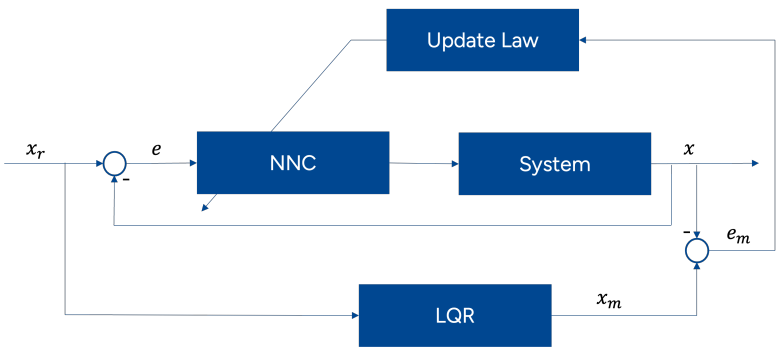
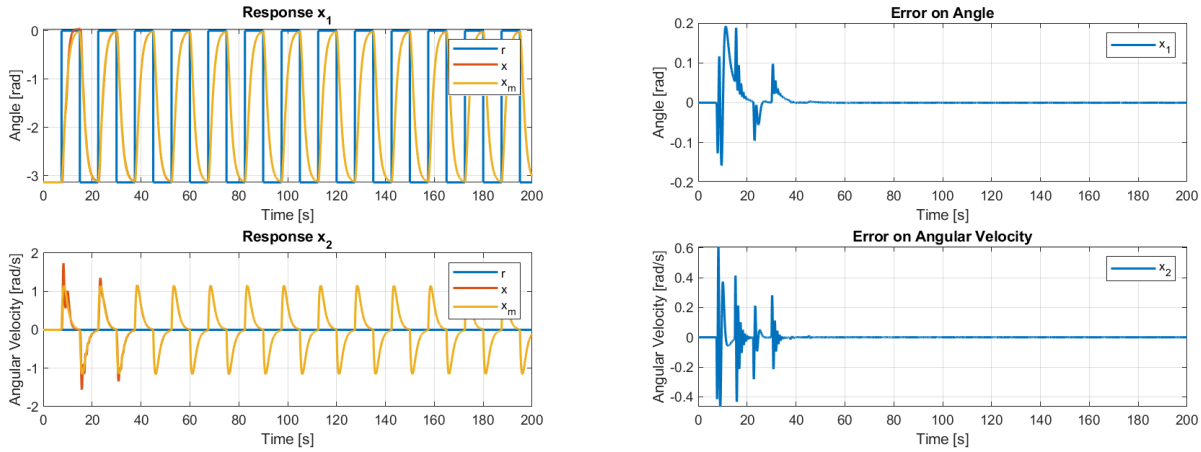



Figure 5.5: MRAC with LQR

The simulation time is 200 seconds with a sampling time of 0.01. The reference signal, meaning the desired position of the pendulum is given by a pulse signal, which alternates between an angle of 0 degrees to $-\pi$, with a periodicity of 15 seconds. In the case of the simulation, the step size is $\Gamma_x = \Gamma_r = 1$. The initial gains $K_x$ and $K_r$ are set to 0.




(a) System response

(b) Error between reference model and system

Figure 5.6: Simulation results MRAC with LQR

The simulation results are displayed in Figure 5.6 and the evolution of the gains is shown in Figure 5.7. Figure 5.6a shows that the reference signal changes for the first time after 7.5 seconds, which will be the point in time from where we judge the behavior. This is because the learning phase starts when the difference between states, and reference model states are non-zero. We observe that in the beginning, the system does not follow the reference signal perfectly, however after about 7.5 seconds after starting to learn, the system output matches the reference model output.

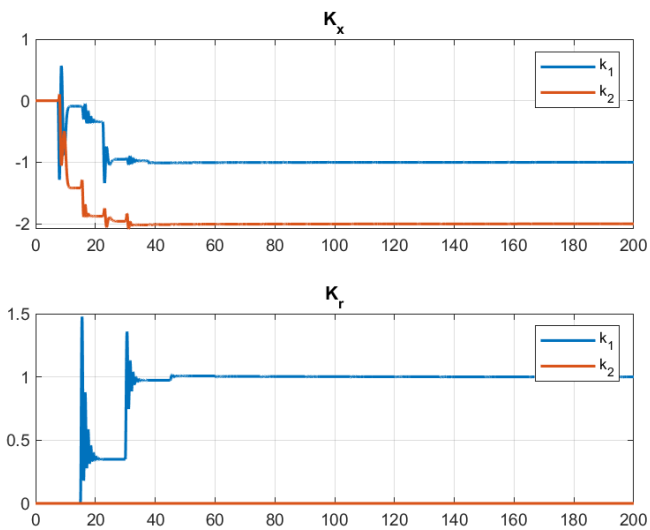



Figure 5.7: MRAC weights over time

When taking a look at the error between the reference model and the system in Figure 5.6b we can observe that the error between both responses converges towards 0 over time. This indicates that the MRAC learns stabilizing gains, that enable the system to track the reference model. Furthermore, if we take a look at the evolution of the weights, as depicted in Figure 5.7, we can observe that the weights converge to values of $K_x = [-1, -2]$, and $K_r = [1, 0]$.

## 5.3 MRACs for Nonlinear Neural Network Controllers

In this section, we will present our simulation for the proposed MRAC strategy for nonlinear NNCs. We carefully construct an example to show that with the proposed schematic we are able to learn the desired parameter. To this end, we choose to use a simple NNC that consists of only one neuron with an activation function, which can be seen as a smooth saturation function, as defined in (4.9). The parameters of the neural network consist of one value that is updated.

For simplicity, we choose the dynamics of the reference model and the system to be equal, and we choose the activation function to be the same. The dynamics of the system and the reference model have a form as defined in equation (4.29) and equation (4.30). The only parameters that are different are $\theta$ and $\theta_m$. Hence, to ensure that the behavior of the reference model and the system are the same, we require $\theta$ to converge towards $\theta_m$. We choose $a_m = a = 10$, $b_m = b = 10$, $\theta_m = 4$, and the activation function saturates at a value of 1. The precision threshold $\epsilon$ is chosen to be computer precision in all our simulations. We compute the weight update as the analytical solution for each time instance. The sampling time for our simulations is $T_s = 0.01$, and the learning rate differs in each simulation to showcase faster and slower convergence.

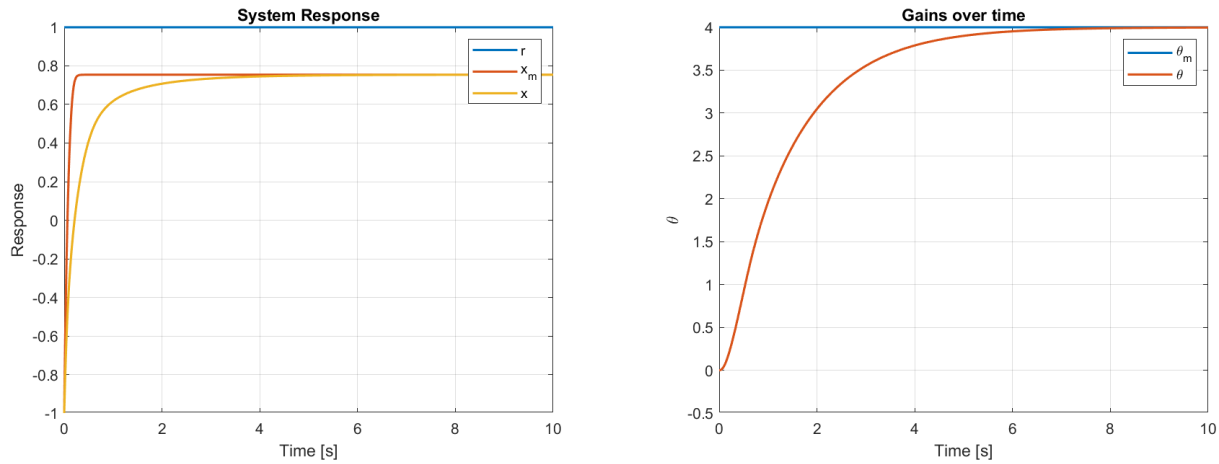
We will analyze the behavior with respect to different reference signals. Note, that we require the reference signal to be differentiable, because $\dot{r}$ is used in the update law. Hence, we will firstly look at the response of a constant signal, secondly a sine signal, and thirdly a smooth pulse signal.

### 5.3.1 Constant Reference Signal

We commence by using a constant reference signal. We choose the system's initial conditions to be at 0. The value of the constant reference signal is 1. This will result in a steady state error for the reference model since we only take a feedback controller into account. However, we will ignore this steady state error, since for us the ability of the controller to learn the behavior of the reference model is of interest. Additionally, we set $\theta_m = 4$ and the initial value of the weight $\theta = 0$. Finally, we simulate the system response for 5 seconds and choose the constant parameter $c = 0.05$.

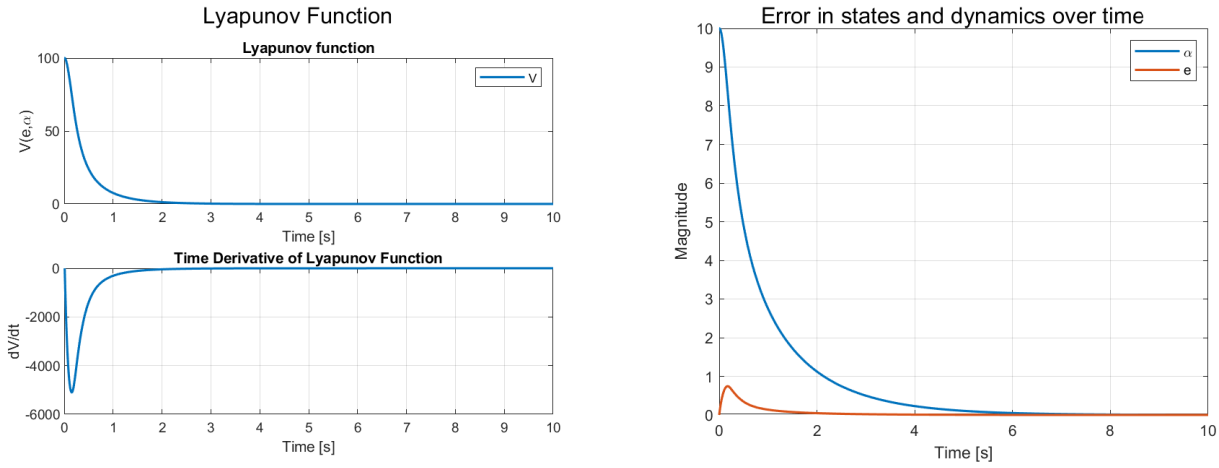
In Figure 5.8a, we observe the steady state error, which is at around 0.2. As aforementioned, we will disregard this error, since we are interested in how well our NNC can learn the behavior of our reference model. To this end, we can observe that the system converges to the reference model within 5 seconds. Furthermore, in Figure 5.8b, we can see the weights of the NNC and the error $e$ are equal to their desired values, $\theta = 4$ and $e = 0$ respectively. In Figure 5.9a, we can see that the Lyapunov function converges to zero over time. Additionally, we can see that the derivate of the Lyapunov function is negative and only reaches zero when the Lyapunov function itself converges to zero. This

indicates that the proposed learning schematic is stable for the desired reference signal, and we can learn the desired parameter.




(a) System response (b) Evolution of gain update

Figure 5.8: Simulation results of nonlinear MRAC with a constant input




(a) Lyapunov function and its time derivate (b) Error in internal states and dynamics of the system

Figure 5.9: Lyapunov function and convergence of the system w.r.t. internal states and dynamics

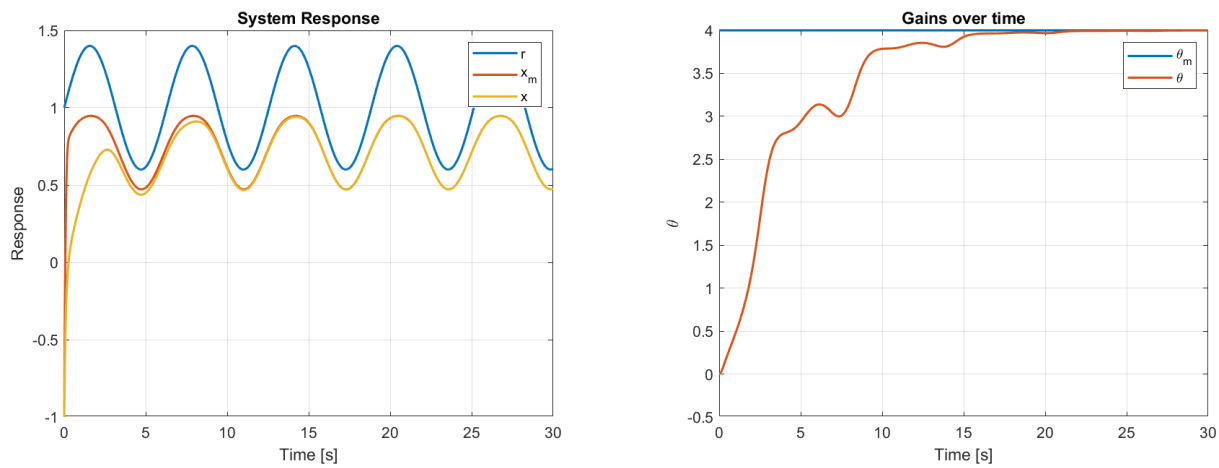
### 5.3.2 Sine Signal

The second reference is a sinusoid signal, with an amplitude of 0.25, to stay in a region that is not affected too much by the saturation function. As before, we choose the initial $\theta = 0$, $\theta_m = 4$, and the initial conditions of both the system and the reference model to be at $-1$. We choose this specific initial condition only to be able to see the convergence in the plots. Finally, we simulate the system response for 30 seconds and choose the constant parameter $c = 0.15$, which corresponds to a less aggressive learning strategy compared to the constant input signal.

Figure 5.10a shows the response of the system. We observe that the system state, and reference

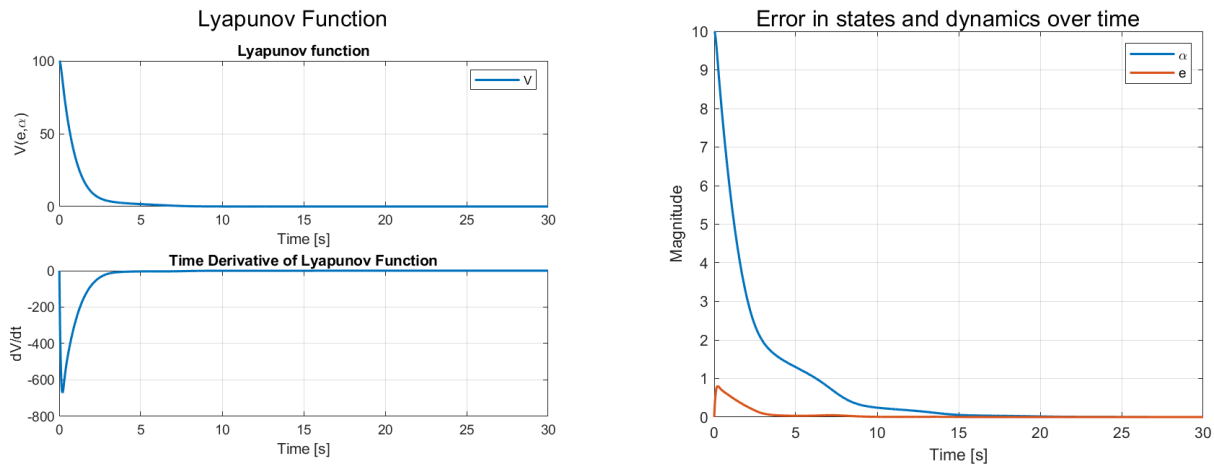
model state both converge towards the desired reference model within around 25 seconds. Simultaneously, the parameter $\theta$ converges to towards the desired value, which can be seen in Figure 5.10b. Even though the parameter converges, we observe oscillations. This behavior is expected, due to the update law. Since the update law is dependent on $\dot{e}_x$, it is dependent on $\dot{r}$ and $\dot{x}$. We observe that whenever $\dot{e}_x$ switches signs and simultaneously $e$ is small, $\dot{\theta}$ changes sign, and hence the parameter starts oscillating. This behavior can be changed thourgh adapting the learning rate. A more aggressive learning strategy will be showcased in the next example.

In Figure 5.11a we can see that the Lyapunov function converges to zero over time. Additionally, we can see that the derivate of the Lyapunov function is negative and only reaches zero when the Lyapunov function itself converges to zero. The results of this simulation indicate again that the proposed learning schematic is stable for the desired reference signal, and we can learn the desired parameter.




(a) System response (b) Evolution of gain update

Figure 5.10: Simulation results of nonlinear MRAC with a sine input




(a) Lyapunov function and its time derivate (b) Error in internal states and dynamics of the system

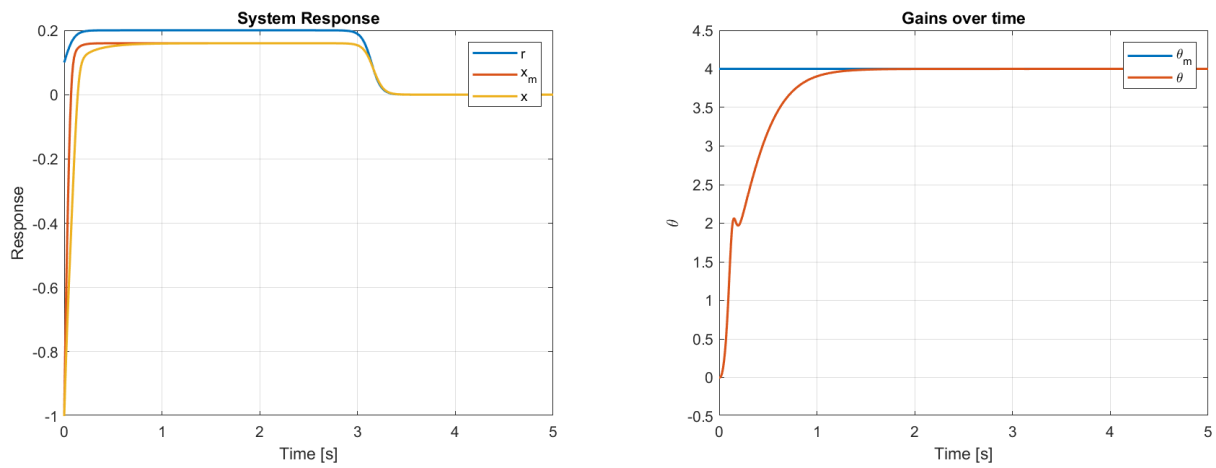
Figure 5.11: Lyapunov function and convergence of the system w.r.t. internal states and dynamics

### 5.3.3 Smooth Pulse Signal

The third and last input signal we will look at is a pulse signal. More specifically, we will use a smooth pulse signal, because we require the reference signal to be continuous since we require its derivative in the update law. We construct the smooth pulse signal, by using a sine function and applying a smooth saturation function, as defined in equation (4.9). We choose the signal to have an amplitude of 0.25 and shift the signal such that its minimum value touches upon 0. As before, we choose the initial conditions of the system and plant to be $-1$. We simulate the system response for 5 seconds and choose the constant parameter $c = 0.005$.

Figure 5.12a shows the response of the system. We can observe that the system converges towards the desired reference model within just under 1 second. The parameter $\theta$ converges to towards the desired value within 2 seconds, which can be seen in Figure 5.12b. Even though the parameter converges, we observe that it oscillates. This behavior is expected, due to the update law. Since the update law is dependent on $\dot{e}_x$, it is dependent on $\dot{r}$ and $\dot{x}$. We observe that whenever $\dot{e}_x$ switches signs and simultaneously $e$ is small, $\dot{\theta}$ changes sign, and hence the parameter starts oscillating. Similarly, as in the case of the sine input, this behavior can be influenced by decreasing the value of $c$.

The Lyapunov function and its derivative, as shown in Figure 5.13a indicate the convergence of the system and the parameters, since $V$ converges to 0 and its derivative is always negative and approach zero when $V$ converges. The results of this simulation indicate that the proposed learning schematic is stable for the desired reference signal, and we are able to learn the desired parameter. The three simulation results strongly indicate that with the proposed update law we are able to learn the desired parameters in the presence of a nonlinear activation function.



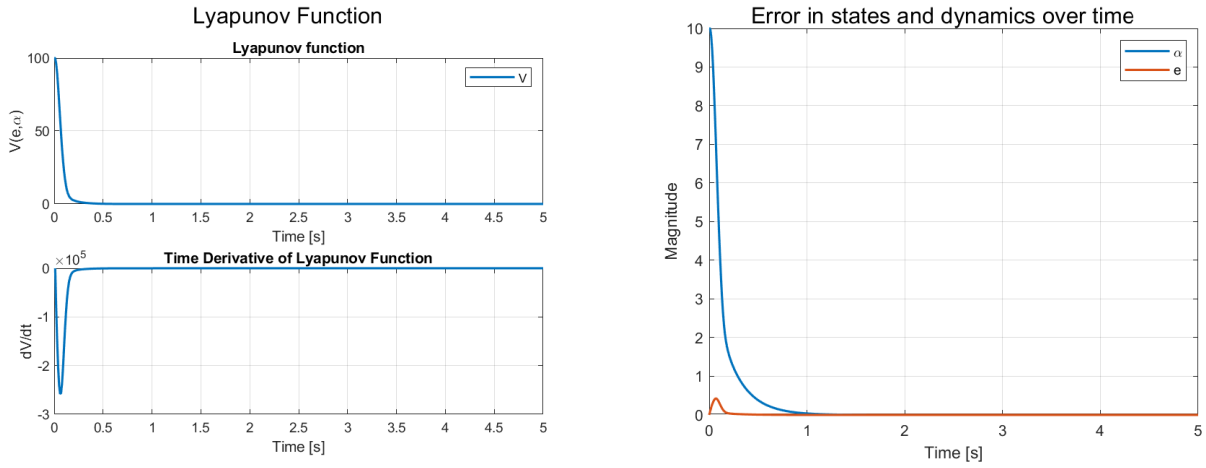

(a) System response

(b) Evolution of gain update

Figure 5.12: Simulation results of nonlinear MRAC with a smooth pulse input

### 5.3.4 Simulation Nonlinear MIMO Systems

In this work we propose an update law in section 4.3.3 for MIMO systems with nonlinear NNCs. To validate the proposed update law, we choose to simulate a double integrator with multiple inputs, as




(a) Lyapunov function and its time derivate

(b) Error in internal states and dynamics of the system

Figure 5.13: Lyapunov function and convergence of the system w.r.t. internal states and dynamics

defined by the matrices in (5.2). Futhermore, since we want to show the convergence of the algorithm, we choose the model reference to be the equal to the system, with different NN weights. More specifically, we choose the inital weights of the network to be 0 for all components.

$$A = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}, \qquad B = \begin{pmatrix} 0 & 0 \\ 1 & -1 \end{pmatrix} \tag{5.2}$$

Note, that this system does not exactly represent the Pendubot model, but can be interpreted as see-saw system with a propellor on each side. We choose this system, because the dynamics are very similar to those of a 2 degree of freedom Pednubot, while allowing us to simulate a system with two states, instead of four, the system would have with an additional translation component, that would provide us with multiple inputs.

It is also important to mention that this method assumes to use a stable model reference. However, since the model reference includes a nonlinearity, prooving stability poses a challenge in itself. The stability of the model reference will be assumed from here hencewoth and not formally proven within the scope of this work. We refer to [35] for further information on the stability analysis of nonlinear systems. Nevertheless, we provide a phase plot in Figure 5.14 for the reader to get an idea of the dynamics.

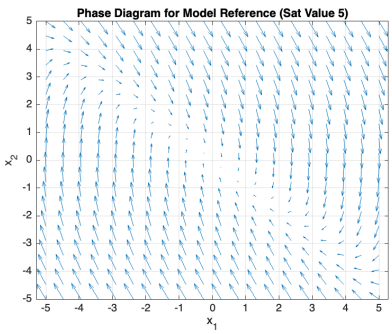
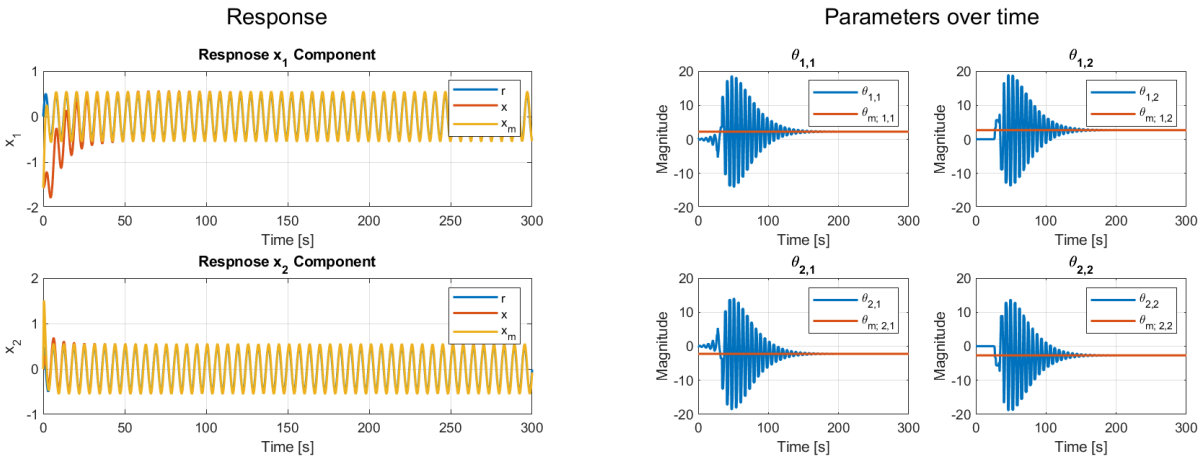



Figure 5.14: Phase plot of model reference dynamics

The chosen reference signal is a sinusoid that oscillates aroung the origin. This can be interpreted as seesawing the system back and forth motion. The initial conditions of the system and the reference model are defined to be $[\frac{\pi}{2}, 0]^T$. Furthermore, we choose $T_s = 0.001s$, $\Gamma = 0.2$, and we simulate the system for 300 seconds.

The system response is shown in Figure 5.15a. We observe that the system response converges towards that of the model reference and it takes less than 50 seconds for the system to exhibit a similar response. Furthermore, in Figure 5.15b it can be observed that the parameter $\theta$ requires around 180 seconds to fully converge to the desired values. Before convergence, the values of the parameters oscillate, which can be explained by the fact that there is not one solution that satisfies the conditions on the update law and the system requires some time to converge towards the desired set of parameters. Again here we can influence the convergence speed by choosing an appropriate value for $\Gamma$.




(a) System response of MIMO System

(b) System response of MIMO System

Figure 5.15: Simulation results of the MIMO nonlinear MRAC

Firgure 5.16 shows the Lyapunov function and its time derivate. It can be observed that the value of the Lyapunov function decreases over time, which corresponds to the error in internal states and the error in dynamics decreasing, which is showcased in Figure 5.17. These responses match our obversations from Figure 5.15a. Furthermore, we observe that the components of the Lyapunov time derivate are not always negative due to the term $dV_Q$, which represents the term $2e^T PAme$. Nevertheless, this does not pose a problem for the simulated system, since the time derivative of the Lyapunov function remains negative, due to the term $dV_\gamma$ being more negative.

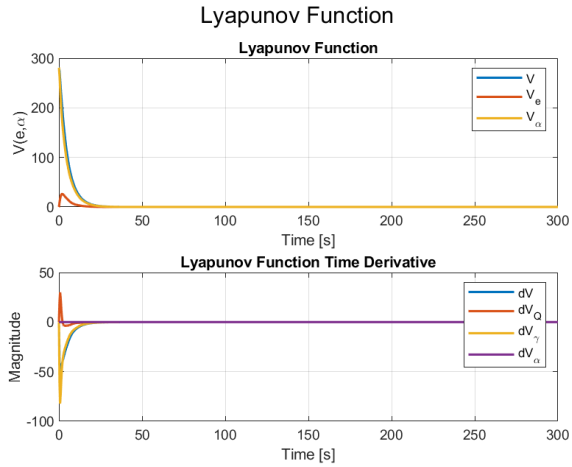
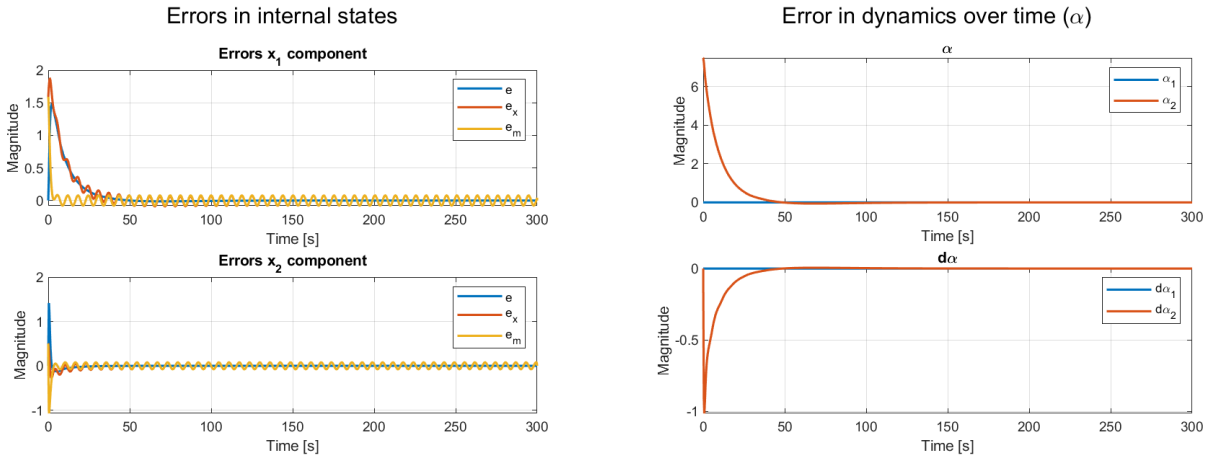



Figure 5.16: Lyapunov function for the simulated MIMO system




(a) Convergence of internal states

(b) Convergence of dynamics

Figure 5.17: Lyapunov function for the simulated MIMO system

# 6 Discussion & Conclusion

In this section, the findings of the study are interpreted and analyzed in the context of the research questions and objectives outlined at the beginning. By examining the results, this discussion aims to highlight our findings, explore their implications, and point out their limitations. Additionally, we conclude this thesis and point out the limitations of the study, and potential directions for future research will be suggested.

## 6.1 Discussion

To discuss our results, we will re-iterate our research questions and answer them directly. We will then discuss their implications and based on our discussion we will propose future work that might arise from our work.

### [RQ1] How do linear MRACs compare to self-tuning PIDs, and PIDNNs, applied to Pendubot, concerning performance and stability?

The self-tuning PID controller converges after just 5 seconds, however, it exhibits an overshoot of more than 100%. In practice, this means that Pendubot does more than one full rotation during the learning phase. However, after the learning phase, the self- tuning PID does not exhibit an overshoot. Opposed to the self-tuning PID controller, the PIDNN exhibits oscillatory behavior during the learning phase. In the first 10 seconds, Pendubot oscillates around the desired state with up to 50% overshoot. However, after the learning phase, the PIDNN can make the system follow the reference signal directly. Furthermore, during simulation we noticed that different learning rates can influence the stability of the system. Hence, the learning rate has to chosen to be small enough for the system to not go unstable. Both the PIDNN, and the self-tuning PID controller converge to a PD controller, which can be seen as a state feedback controller. The linear MRAC learns similarly quickly as the self-tuning PID controller, however, it does not exhibit an overshoot as severe. The angle only overshoots by 1.22%. After the learning phase, the controller exhibits good reference tracking performance.

When comparing the performance of the three approaches, we can see that they are all able to stabilize the system in an upright position. During the learning phase, the PIDNN struggles the most. The linear MRAC exhibits the best behavior when compared to the speed of learning and the magnitude of the overshoot during the learning phase.

Furthermore, during learning, we cannot guarantee the stability of the self-tuning PID and the PIDNN. To this end, we require a posteriori stability analysis. This means that in every update step, we would have to extract the new weights and verify their stability, through for example finding an appropriate Lyapunov function. Here we can see a clear advantage of the linear MRAC. Opposed to the self-tuning PID and the PIDNN, the linear MRAC learns stabilizing weights by definition. In every update step, we can guarantee that the set of weights is stabilizing. Finally, both the fast convergence and the stability properties of the linear MRAC render the control strategy to be better for the proposed system.

### [RQ2] How can we develop stable learning algorithms, based on MRACs, controlled by nonlinear NNCs?

In this thesis, we proposed a Lyapunov-based learning algorithm for nonlinear MRACs, by deriving an update law based on the logic of linear MRACs, as proposed in many works before.

Firstly, we worked out the update law for a scalar system, as defined in equation (4.37). For validation, a nonlinear MRAC for a first-order system is simulated with three different reference signals, namely 1) a constant input signal, 2) a sine input signal, and 3) a smooth pulse input signal. The simulation results show that the proposed update law works well and that the controlled system converges to the model reference. This is observed in two points, namely the internal states of the system and the dynmics of the system converge to those of the model reference. Furthermore, we observe that we can influence the speed of convergence by choosing the factor $c$. To increase the speed, we decrease $c$ and vice versa.

Another interesting observation we made is that the update law is susceptible to numerical instability, which can be observed by spikes in $\dot{\theta}$, hence leading the controller and Lyapunov function to spike as well. This is due to using discrete numerical methods to approximate the real continuous time derivatives within the update law. When the sampling time is too big, the approximation of the derivates is not precise enough, which leads the controller to update the parameter $\theta$, such that the condition on the Lyapunov function, namely $e = -\dot{\alpha}(e_x, \theta)$, is not satisfied. This leads to a discrepancy between the numerical and analytical solution. However, this behavior can be prevented by decreasing the sampling time of the simulation, leading to a more precise approximation of the derivatives. Simultaneously, decreasing the sampling time will also yield more precise integrals, hence the overall accuracy of the simulation will improve.

Additionally to an update law for a scalar system, we proposed an update law for MIMO systems, as defined in (4.46). Similarly, we validate this update law through simulating a seasaw system, which has similar dynamics to a two degree of freedom Pendubot model, with a translational component around the origin. The controller has four neurons with a sigmoid saturation function at every neuron. Again, we choose the system to be equal to the model reference, hence the only difference lies in the controller parameters. The expected result is the convergence of the controller to the one used withn the reference model, because the value of the Lyapunov function can only be zero if the matching conditions are satisfied, meaning the the internal states and dynamics of the model reference are equal to those of the controlled system. These conditions can only be satisfied, if there exists a combination of NN parameters that make the controlled system equal to model reference.

In our simulations these conditions are satisfied, since NN parameters exist that make the model reference and the system exactly the same and the expected bahavior can be observed. Additionally, we observe that the value of the Lyapunov function converges to zero, which in turn implies that the internal states and the dynamics have converged to one another and the network stops updating. This observation aligns with our expectations.

Given our simulations of the case when the matching conditions are satified, it is interesting to discuss what is expected to happen when they are not satisfied. The simplest example is to think of an unsaturated LQR controller, which we aim to approximate with a NN using nonlinear activation

functions. Since there is no combination of parameters that make the controlled system and the model reference equal to one another, the update law will keep updating the NN parameters and should continue oscillating forever, assuming that the simulations do not contain errors caused by discrete numerical approximations. To validate this experimental setup, we would require to develop a discrete-time update law. This will be part of future research.

## 6.2 Conclusion

This thesis investigates the performance and stability of various control strategies applied to the Pendubot, specifically focusing on self-tuning PID controllers, PIDNNs, and linear MRACs. Additionally, we propose a nonlinear MRAC update schematic and provide proof of concept simulations.

Firstly, the comparison of control strategies (RQ1) shows distinct differences in performance and stability. The self-tuning PID controller converges rapidly within 5 seconds but suffers from significant overshoot (more than 100%) during the learning phase. However, it stabilizes without overshoot post-learning. In contrast, the PIDNN controller exhibits oscillatory behavior during the initial 10 seconds with a 50% overshoot but achieves good reference tracking performance afterwards. The linear MRAC, while learning quickly like the self-tuning PID, demonstrates minimal overshoot (1.22%) and excellent reference tracking after the learning phase. Overall, the linear MRAC shows superior performance in terms of learning speed and reduced overshoot, making it the most effective control strategy for the Pendubot among the three evaluated methods. Due to its superiority we extend the concept to NMRACs.

Secondly, in terms of the development of stable learning algorithms (RQ2), based on the principles of linear MRACs, a new update law for a system with a nonlinear NNC was derived and validated through simulations for a scalar and a MIMO system.

The simulation results of the scalar system indicate that the proposed update law successfully enables the system to learn the desired parameters across different reference signals. The speed of convergence can be controlled by adjusting the factor $c$, with smaller values leading to faster convergence. Additionally, this work identifies numerical instability as a potential issue. This instability can be reduced by lowering the sampling time, thus enhancing the precision of integral and derivative approximations, improving overall simulation accuracy. The simulations results of the MIMO system show that with the proposed learning algorithm the NN converges to the desired values. Just like in the first order case, choice of $\Gamma$ influences the convergence speed. Finally, we want to point out that due to the choice of the Lyapunov function, it has the property that it can only be zero when the internal states and the dynamics of the controlled system and model reference are exactly equal. This ensures the convergence of the proposed learning algorithm, under the assumption that the matching conditions are satisfied.

Overall, this thesis highlights the promising potential of linear MRACs for achieving rapid convergence and minimal overshoot in controlling Pendubot. Based on this knowledge and the logic of linear MRAC, we developed a Lyapunov-based nonlinear MRAC. We provided proof of concept simulations for the proposed update law, which emphasizes its potential. Therefore, this work constitutes a solid foundation for further research on nonlinear MRAC. The next and final chapter will descuss possible

extensions of this work.

## 6.3 Outlook

This section aims to provide a forward-looking perspective on the potential future developments and applications of the research presented in this thesis. Building on the findings and discussions from earlier sections, the outlook will explore possible directions for further research, advancements in the field, and practical implementations of the study's results.

### 6.3.1 Partially Uknown Dynamics

The simulations presented in this thesis are all based on the assumption of full knowledge of the system dynamics. However, in reality this assumption is rarely fulfilled. Hence, future work should focus on integrating the possiblity of not having to know the full dynamics of the system. Wihthin the scope of this project we investigated typical methods for getting rid of the control input dynamics, as proposed in [40, 6, 61]. Unfortunately, following these approaches did not yield the desired results. This leads us to believe that other methods should be investigated.

### 6.3.2 Investigating the Decrease Condition

In our simulations, we specifically choose an example where these conditions are statisfied. However, the proposed Lyapunov candidate does not guarantee that its time derivate will be negative for all systems. To shed more light on the properties of this Lyapunov candidate, future research should investigate when the decrease condition is violated and when it is not violated. This investigation can be conducted globally but also locally. This could yield interesting insights into the inner workings of adaptive control and its stability properties.

### 6.3.3 Mitigation of Steady-State Errors for the Nonlinear MRACs

In this work, we focus on designing an update law, based on the logic of MRACs. Hence, we focus on the ability to learn the parameters of a nonlinear reference model. This resulted in ignoring steady-state errors that occur, for example in the case of the step input and the pulse signal. However, in practice, we do not want our system to have a steady-state error. Therefore, the question arises how to mitigate this steady-state error. Typically, by introducing an integral action, we can compensate for this steady-state error. However, by doing so the equations for our update might change. Therefore, an interesting investigation would be to investigate how the integral action is included into linear MRACs, how this changes the required matching conditions, and how this concept could be transferred to MRAC including nonlinearities to mitigate steady-state errors.

### 6.3.4 Tackling Numerical Instability

As discussed in Section 6.1 the update law for nonlinear MRACs for first-order systems, as proposed in equation (4.37), is numerically not very stable. We mitigate this problem, by decreasing the

sampling time of the simulation. However, an update law, based on a discrete-time derivation may yield in a more stable version. Therefore, investigating a pure discrete-time update law and comparing its numerical stability to the update law derived from the continuous case would be an interesting extension of this work.

### 6.3.5 Implementation on Hardware

The results achieved in this thesis are purely through simulation. However, in reality there is always a discrepency between hardware implementation and simulation. Hence, in the next step these update mechanisms should be implemented. We suggest to commence with implementing the proposed MIMO update law to a seasaw system, as the dynamics are described by matrices (5.2) and a stabilizing controller as a model reference is already known. This system can be used to immediately show that the learning algorithms also works in practice.

### 6.3.6 Approximating Model Predictive Controllers

Finally, in this work we made use of a saturated LQR controller as model reference for the validation of the MIMO update law, which does not allow constraints on the optimization problem. This is where Model Predivtive Control (MPC) comes into play, with which it is possible to add a multitude of constraints to the optimization problem, which yields an optimal solution. Despite these advantages, MPC has significant drawbacks, such as having to either solve an optimzation problem online or store a piecewise affine function, where the memory usage grows with the number of polyhedral regions. This can become problematic for applications that require an operation at high frequencies or do not have a large amount of storage available. This problem can be circumvented by using a NN to approxmiate an MPC, which is then extremely fast in evaluation and uses much less storage. Additionally, Karg and Lucia show that an MPC controller can be efficiently respresented as a ReLU network [32]. Hence, we believe that learning an approximate MPC controller with a ReLU NN, while guaranteeing stability is a very promising research avenue that should be explored.

# Bibliography


[1] E. J. Adkins and L. W. Taylor. *Adaptive flight control systems - pro and con.* Tech. rep. NASA-TM-X-56008. NTRS Author Affiliations: NASA Flight Research Center NTRS Document ID: 19640017347 NTRS Research Center: Legacy CDMS (CDMS). Apr. 1964. URL: https://ntrs.nasa.gov/citations/19640017347 (visited on 04/23/2024).

[2] M. S. Ahmed. "Neural net based MRAC for a class of nonlinear plants". In: *Neural Networks* 13.1 (Jan. 2000), pp. 111–124. ISSN: 0893-6080. DOI: 10.1016/S0893-6080(99)00082-9. URL: https://www.sciencedirect.com/science/article/pii/S0893608099000829 (visited on 04/24/2024).

[3] Thamer Albahkali, Ranjan Mukherjee, and Tuhin Das. "Swing-Up Control of the Pendubot: An Impulse–Momentum Approach". In: *IEEE Transactions on Robotics* 25.4 (Aug. 2009). Conference Name: IEEE Transactions on Robotics, pp. 975–982. ISSN: 1941-0468. DOI: 10.1109/TRO.2009.2022427. URL: https://ieeexplore.ieee.org/document/5067337/?arnumber=5067337 (visited on 07/16/2024).

[4] Karl Aström. "History of Adaptive Control". en. In: *Encyclopedia of Systems and Control.* Ed. by John Baillieul and Tariq Samad. London: Springer, 2014, pp. 1–9. ISBN: 978-1-4471-5102-9. DOI: 10.1007/978-1-4471-5102-9_120-1. URL: https://doi.org/10.1007/978-1-4471-5102-9_120-1 (visited on 04/19/2024).

[5] K. J. Åström. "Theory and applications of adaptive control—A survey". In: *Automatica* 19.5 (Sept. 1983), pp. 471–486. ISSN: 0005-1098. DOI: 10.1016/0005-1098(83)90002-X. URL: https://www.sciencedirect.com/science/article/pii/000510988390002X (visited on 04/19/2024).

[6] Karl Johan Åström and Björn Wittenmark. *Adaptive Control.* en. Courier Corporation, Jan. 2008. ISBN: 978-0-486-46278-3.

[7] Errachdi Ayachi. "A new algorithm for MRAC method using a neural variable learning rate". In: *INTERNATIONAL JOURNAL of NEURAL NETWORKS and ADVANCED APPLICATIONS* Volume 1 (Jan. 2014), pp. 1–6.

[8] Ramashis Banerjee, Naiwrita Dev, and Ujjwal Mondal. "MRAC BasedApproachforStabilization of Single Link InvertedPendulum". In: *2018 International Conference on Current Trends towards Converging Technologies (ICCTCT).* Mar. 2018, pp. 1–4. DOI: 10.1109/ICCTCT.2018.8550846. URL: https://ieeexplore.ieee.org/abstract/document/8550846 (visited on 07/16/2024).

[9] Micha Bosshart. "Application of Machine Learning to Self-Tuning Controllers". English. Bachelor Thesis. ETH Zürich, July 2021.

[10] Micha Bosshart, Guillaume Ducard, and Christopher Onder. "Comparison Between Two PID Neural Network Controller Approaches - Simulation and Discussion". In: *2021 International Conference on Control, Automation and Diagnosis (ICCAD).* ISSN: 2767-9896. Nov. 2021, pp. 1–6. DOI: 10.1109/ICCAD52417.2021.9638766. URL: https://ieeexplore.ieee.org/abstract/document/9638766 (visited on 02/28/2024).

[11] Olfa Boubaker. "The inverted pendulum: A fundamental benchmark in control theory and robotics". In: *International Conference on Education and e-Learning Innovations*. July 2012, pp. 1–6. DOI: 10.1109/ICEELI.2012.6360606. URL: https://ieeexplore.ieee.org/document/6360606/?arnumber=6360606 (visited on 07/15/2024).

[12] Gregorio Carughi, Guillaume Ducard, and Christopher Onder. "Online Neural-Network Learning and Model Predictive Control Applied to a Tilt-Rotor Unmanned Aerial Vehicle". en. In: *2022 IEEE 17th International Conference on Control & Automation (ICCA)*. Naples, Italy: IEEE, June 2022, pp. 31–37. ISBN: 978-1-66549-572-1. DOI: 10.1109/ICCA54724.2022.9831923. URL: https://ieeexplore.ieee.org/document/9831923/ (visited on 05/06/2024).

[13] S. Cong and Y. Liang. "PID-Like Neural Network Nonlinear Adaptive Control for Uncertain Multivariable Motion Control Systems". en. In: *IEEE Transactions on Industrial Electronics* 56.10 (Oct. 2009), pp. 3872–3879. ISSN: 0278-0046. DOI: 10.1109/TIE.2009.2018433. URL: http://ieeexplore.ieee.org/document/4812095/ (visited on 05/23/2024).

[14] M.A. Duarte and K.S. Narendra. "Combined direct and indirect approach to adaptive control". en. In: *IEEE Transactions on Automatic Control* 34.10 (Oct. 1989), pp. 1071–1075. ISSN: 00189286. DOI: 10.1109/9.35278. URL: http://ieeexplore.ieee.org/document/35278/ (visited on 04/29/2024).

[15] Manuel A. Duarte-Mermoud, Franklin A. Rojo, and Ricardo Pérez. "Experimental evaluation of combined model reference adaptive controller in a pH regulation process". en. In: *International Journal of Adaptive Control and Signal Processing* 16.2 (2002). _eprint: https://onlinelibrary.wiley.com/doi/p pp. 85–106. ISSN: 1099-1115. DOI: 10.1002/acs.674. URL: https://onlinelibrary.wiley.com/doi/abs/10.1002/acs.674 (visited on 04/29/2024).

[16] Manuel A. Duarte-Mermoud, Jaime S. Rioseco, and Rodrigo I. González. "Control of longitudinal movement of a plane using combined model reference adaptive control". In: *Aircraft Engineering and Aerospace Technology* 77.3 (Jan. 2005). Publisher: Emerald Group Publishing Limited, pp. 199–213. ISSN: 0002-2667. DOI: 10.1108/00022660510597223. URL: https://doi.org/10.1108/00022660510597223 (visited on 04/29/2024).

[17] Guillaume Ducard. "Digital Control". Lecture Notes. Polytech Nice-Sophia, Université Cote d'Azur, 2022.

[18] I. Fantoni, R. Lozano, and M.W. Spong. "Energy based control of the Pendubot". In: *IEEE Transactions on Automatic Control* 45.4 (Apr. 2000). Conference Name: IEEE Transactions on Automatic Control, pp. 725–729. ISSN: 1558-2523. DOI: 10.1109/9.847110. URL: https://ieeexplore.ieee.org/document/847110/?arnumber=847110 (visited on 07/16/2024).

[19] Mahyar Fazlyab, Manfred Morari, and George J. Pappas. "Safety Verification and Robustness Analysis of Neural Networks via Quadratic Constraints and Semidefinite Programming". en. In: *IEEE Transactions on Automatic Control* 67.1 (Jan. 2022), pp. 1–15. ISSN: 0018-9286, 1558-2523, 2334-3303. DOI: 10.1109/TAC.2020.3046193. URL: https://ieeexplore.ieee.org/document/9301422/ (visited on 05/23/2024).

[20] Mahyar Fazlyab et al. "Efficient and Accurate Estimation of Lipschitz Constants for Deep Neural Networks". In: *Advances in Neural Information Processing Systems*. Vol. 32. Curran Associates, Inc., 2019. URL: https://proceedings.neurips.cc/paper_files/paper/2019/hash/95e1533eb1b20a97777749fb94fdb944-Abstract.html (visited on 05/23/2024).

[21] Hao Fu et al. "MRAC for unknown discrete-time nonlinear systems based on supervised neural dynamic programming". In: *Neurocomputing* 384 (Apr. 2020), pp. 130–141. ISSN: 0925-2312. DOI: 10.1016/j.neucom.2019.12.023. URL: https://www.sciencedirect.com/science/article/pii/S0925231219317254 (visited on 07/09/2024).

[22] Aman Goel, Rahul Kumar, and Shiv Narayan. "Design of MRAC augmented with PID controller using genetic algorithm". In: *2016 IEEE 1st International Conference on Power Electronics, Intelligent Control and Energy Systems (ICPEICES)*. July 2016, pp. 1–5. DOI: 10.1109/ICPEICES.2016.7853149. URL: https://ieeexplore.ieee.org/document/7853149/?arnumber=7853149 (visited on 07/16/2024).

[23] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. "Universal approximation of an unknown mapping and its derivatives using multilayer feedforward networks". In: *Neural Networks* 3.5 (Jan. 1990), pp. 551–560. ISSN: 0893-6080. DOI: 10.1016/0893-6080(90)90005-6. URL: https://www.sciencedirect.com/science/article/pii/0893608090900056 (visited on 05/16/2024).

[24] Robert D. Howe. "Tactile sensing and control of robotic manipulation". In: *Advanced Robotics* 8.3 (Jan. 1993). Publisher: Taylor & Francis _eprint: https://doi.org/10.1163/156855394X00356, pp. 245–261. ISSN: 0169-1864. DOI: 10.1163/156855394X00356. URL: https://doi.org/10.1163/156855394X00356 (visited on 04/02/2024).

[25] Feni Isdaryani, Rudi Salam, and Feriyonika Feriyonika. "Design and Implementation of Two-Wheeled Robot Control Using MRAC". en. In: *Journal of Telecommunication, Electronic and Computer Engineering (JTEC)* 13.3 (Sept. 2021). Number: 3, pp. 25–30. ISSN: 2289-8131. URL: https://jtec.utem.edu.my/jtec/article/view/6108 (visited on 07/18/2024).

[26] S. Jagannathan and F.L. Lewis. "Multilayer discrete-time neural-net controller with guaranteed performance". en. In: *IEEE Transactions on Neural Networks* 7.1 (Jan. 1996), pp. 107–130. ISSN: 10459227. DOI: 10.1109/72.478396. URL: http://ieeexplore.ieee.org/document/478396/ (visited on 04/29/2024).

[27] Yiming Jiang et al. "A Brief Review of Neural Networks Based Learning and Control and Their Applications for Robots". en. In: *Complexity* 2017 (Oct. 2017). Publisher: Hindawi, e1895897. ISSN: 1076-2787. DOI: 10.1155/2017/1895897. URL: https://www.hindawi.com/journals/complexity/2017/1895897/ (visited on 02/12/2024).

[28] Anton Johansson et al. "Improved Spectral Norm Regularization for Neural Networks". In: *Modeling Decisions for Artificial Intelligence*. Ed. by Vicenç Torra and Yasuo Narukawa. Cham: Springer Nature Switzerland, 2023, pp. 181–201. ISBN: 978-3-031-33498-6.

[29] Mikael Johansson. *Lecture Notes: Linear quadratic and model predictive control*. en. URL: https://people.kth.se/~mikaelj/documents/EL2700_Lecture_Notes_20240311.pdf.

[30] Girish Joshi and Girish Chowdhary. *Deep Model Reference Adaptive Control*. en. arXiv:1909.08602 [cs, eess]. Sept. 2019. URL: http://arxiv.org/abs/1909.08602 (visited on 06/19/2024).

[31] Seul Jung, Hyun-Taek Cho, and T. C. Hsia. "Neural Network Control for Position Tracking of a Two-Axis Inverted Pendulum System: Experimental Studies". en. In: *IEEE Transactions on Neural Networks* 18.4 (July 2007), pp. 1042–1048. ISSN: 1045-9227. DOI: 10.1109/TNN.2007.899128. URL: http://ieeexplore.ieee.org/document/4267711/ (visited on 02/12/2024).

[32] Benjamin Karg and Sergio Lucia. "Efficient Representation and Approximation of Model Predictive Control Laws via Deep Learning". In: *IEEE Transactions on Cybernetics* 50.9 (2020), pp. 3866–3878. DOI: 10.1109/TCYB.2020.2999556.

[33] S. Karnalasadan and A.A. Ghandakly. "A neural network based intelligent model reference adaptive controller". en. In: *2004 IEEE International Conference onComputational Intelligence for Measurement Systems and Applications, 2004. CIMSA.* Boston, MA, USA: IEEE, 2004, pp. 174–179. ISBN: 978-0-7803-8341-8. DOI: 10.1109/CIMSA.2004.1397257. URL: http://ieeexplore.ieee.org/document/1397257/ (visited on 06/21/2024).

[34] Nils Keller, Jan Schilliger, and Yannick Schnider. *Predictive Control of a Two-Wheeled Balancing Robot: Lab Practice Control Systems - Experiment Script*. en. Tech. rep.

[35] Hassan K. Khalil. *Nonlinear Systems*. en. Google-Books-ID: qiBuQgAACAAJ. Prentice Hall, 1996. ISBN: 978-0-13-228024-2.

[36] Dai Kinoshita, Koichi Otsuka, and Itaru Matsumoto. "Swing up control of the pendubot with restricted actuator movement angle using energy-based methods". In: *4th international conference on machining, materials and mechanical technologies (IC3MT)*. Vol. 139. Advances in science and technology. Trans Tech Publications Ltd, Mar. 2024, pp. 69–76. DOI: 10.4028/p-2pl5aQ.

[37] Milan Korda. *Stability and performance verification of dynamical systems controlled by neural networks: algorithms and complexity*. arXiv:2102.02273 [math]. Sept. 2022. DOI: 10.48550/arXiv.2102.02273. URL: http://arxiv.org/abs/2102.02273 (visited on 02/21/2024).

[38] Milan Korda and Colin N. Jones. "Stability and performance verification of optimization-based controllers". In: *Automatica* 78 (Apr. 2017), pp. 34–45. ISSN: 0005-1098. DOI: 10.1016/j.automatica.2016.12.008. URL: https://www.sciencedirect.com/science/article/pii/S0005109816305003 (visited on 05/23/2024).

[39] E. Lavretsky. "Combined/Composite Model Reference Adaptive Control". en. In: *IEEE Transactions on Automatic Control* 54.11 (Nov. 2009), pp. 2692–2697. ISSN: 0018-9286, 1558-2523. DOI: 10.1109/TAC.2009.2031580. URL: http://ieeexplore.ieee.org/document/5289973/ (visited on 04/24/2024).

[40] Eugene Lavretsky and Kevin A. Wise. *Robust and Adaptive Control: With Aerospace Applications*. en. Advanced Textbooks in Control and Signal Processing. London: Springer London, 2013. ISBN: 978-1-4471-4395-6 978-1-4471-4396-3. DOI: 10.1007/978-1-4471-4396-3. URL: https://link.springer.com/10.1007/978-1-4471-4396-3 (visited on 05/31/2024).

[41] F.L. Lewis, K. Liu, and A. Yesildirek. "Neural net robot controller with guaranteed tracking performance". In: *IEEE Transactions on Neural Networks* 6.3 (May 1995). Conference Name: IEEE Transactions on Neural Networks, pp. 703–715. ISSN: 1941-0093. DOI: 10.1109/72.377975. URL: https://ieeexplore.ieee.org/document/377975 (visited on 05/13/2024).

[42] James M. Longuski, José J. Guzmán, and John E. Prussing. *Optimal control with aerospace applications*. eng. 1st ed. 2014. Space technology library ; 32. New York, NY: Springer New York, 2014. ISBN: 1-4614-8945-8.

[43] Iven M. Y. Mareels et al. "Revisiting the Mit Rule for Adaptive Control". In: *IFAC Proceedings Volumes*. 2nd IFAC Workshop on Adaptive Systems in Control and Signal Processing 1986, Lund, Sweden, 30 June-2 July 1986 20.2 (July 1987), pp. 161–166. ISSN: 1474-6670. DOI: 10.1016/S1474-6670(17)55954-6. URL: https://www.sciencedirect.com/science/article/pii/S1474667017559546 (visited on 04/19/2024).

[44] Ibrahim M Mehedi et al. "Underactuated rotary inverted pendulum control using robust generalized dynamic inversion". In: *Journal of Vibration and Control* 26.23-24 (2020), pp. 2210–2220. DOI: 10.1177/1077546320916022. URL: https://doi.org/10.1177/1077546320916022.

[45] Chaoxu Mu, Ding Wang, and Haibo He. "Novel iterative neural dynamic programming for data-based approximate optimal control design". In: *Automatica* 81 (July 2017), pp. 240–252. ISSN: 0005-1098. DOI: 10.1016/j.automatica.2017.03.022. URL: https://www.sciencedirect.com/science/article/pii/S0005109817301450 (visited on 07/09/2024).

[46] Kumpati S. Narendra and Manuel A. Duarte. "Robust Adaptive Control Using Combined Direct and Indirect Methods". en. In: *1988 American Control Conference*. Atlanta, GA, USA: IEEE, June 1988, pp. 2429–2434. DOI: 10.23919/ACC.1988.4790133. URL: https://ieeexplore.ieee.org/document/4790133/ (visited on 04/29/2024).

[47] Huynh Vinh Nghi, Dinh Phuoc Nhien, and Dang Xuan Ba. "A LQR Neural Network Control Approach for Fast Stabilizing Rotary Inverted Pendulums". en. In: *International Journal of Precision Engineering and Manufacturing* 23.1 (Jan. 2022), pp. 45–56. ISSN: 2234-7593, 2005-4602. DOI: 10.1007/s12541-021-00606-x. URL: https://link.springer.com/10.1007/s12541-021-00606-x (visited on 07/16/2024).

[48] Griffin Norris, Guillaume J. J. Ducard, and Christopher Onder. "Neural Networks for Control: A Tutorial and Survey of Stability-Analysis Methods, Properties, and Discussions". en. In: *2021 International Conference on Electrical, Computer, Communications and Mechatronics Engineering (ICECCME)*. Mauritius, Mauritius: IEEE, Oct. 2021, pp. 1–6. ISBN: 978-1-66541-262-9. DOI: 10.1109/ICECCME52200.2021.9590912. URL: https://ieeexplore.ieee.org/document/9590912/ (visited on 05/06/2024).

[49] Jeb S. Orr, Irving C. Statler, and Immanuel Barshi. "The X-15 3-65 Accident: An Aircraft Systems and Flight Control Perspective". en. In: *Space Safety is No Accident*. Ed. by Tommaso Sgobba and Isabelle Rongier. Cham: Springer International Publishing, 2015, pp. 249–257. ISBN: 978-3-319-15981-2 978-3-319-15982-9. DOI: 10.1007/978-3-319-15982-9_30. URL: https://link.springer.com/10.1007/978-3-319-15982-9_30 (visited on 04/23/2024).

[50] Selahattin Ozcelik et al. "Control of an Inverted Pendulum Using Direct Model Reference Adaptive Control". In: *IFAC Proceedings Volumes*. IFAC Conference on Control of Industrial Systems "Control for the Future of the Youth", Belfort, France, 20-22 May 30.6 (May 1997), pp. 585–590. ISSN: 1474-6670. DOI: 10.1016/S1474-6670(17)43427-6. URL: https://www.sciencedirect.com/science/article/pii/S1474667017434276 (visited on 08/02/2024).

[51] H.D. Patino and D. Liu. "Neural network-based model reference adaptive control system". en. In: *IEEE Transactions on Systems, Man and Cybernetics, Part B (Cybernetics)* 30.1 (Feb. 2000), pp. 198–204. ISSN: 10834419. DOI: 10.1109/3477.826961. URL: http://ieeexplore.ieee.org/document/826961/ (visited on 04/24/2024).

[52] Dixit Sethi et al. "Design of neural network based MRAC". en. In: *International Journal of System Control and Information Processing* 2.4 (Jan. 2019). URL: https://doi.org/10.1504/IJSCIP.2018.097138.

[53] Suresh P. Sethi and Gerald L. Thompson. "Applications of Mathematical Control Theory to Finance: Modeling Simple Dynamic Cash Balance Problems". In: *The Journal of Financial and Quantitative Analysis* 5.4/5 (1970). Publisher: Cambridge University Press, pp. 381–394. ISSN: 00221090, 17566916. DOI: 10.2307/2330038. URL: http://www.jstor.org/stable/2330038 (visited on 11/24/2023).

[54] B. Shackcloth and R. L. But Chart. "Synthesis of Model Reference Adaptive Systems by Liapunov's Second Method". In: *IFAC Proceedings Volumes*. 2nd IFAC Symposium on the Theory of Self-Adaptive Control Systems, Teddington, UK, September 14-17, 1965 2.2 (Sept. 1965), pp. 145–152. ISSN: 1474-6670. DOI: 10.1016/S1474-6670(17)69028-1. URL: https://www.sciencedirect.com/science/article/pii/S1474667017690281 (visited on 04/24/2024).

[55] Sabrine Slama, Ayachi Errachdi, and Mohamed Benrejeb. "A Neural Model Reference Adaptive Controller Algorithm for Nonlinear Systems". en. In: ().

[56] Sabrine Slama, Ayachi Errachdi, and Mohamed Benrejeb. "Model reference adaptive control for MIMO nonlinear systems using RBF neural networks". en. In: *2018 International Conference on Advanced Systems and Electric Technologies (IC_ASET)*. Hammamet: IEEE, Mar. 2018, pp. 346–351. ISBN: 978-1-5386-4449-2. DOI: 10.1109/ASET.2018.8379880. URL: https://ieeexplore.ieee.org/document/8379880/ (visited on 06/19/2024).

[57] J.-J. E. Slotine and Weiping Li. *Applied nonlinear control*. en. Englewood Cliffs, N.J: Prentice Hall, 1991. ISBN: 978-0-13-040890-7.

[58] Jean-jacques Slotine and Weiping Li. "Adaptive robot control: A new perspective". en. In: *26th IEEE Conference on Decision and Control*. Los Angeles, California, USA: IEEE, Dec. 1987, pp. 192–198. DOI: 10.1109/CDC.1987.272740. URL: http://ieeexplore.ieee.org/document/4049250/ (visited on 04/29/2024).

[59] Jean-Jacques E. Slotine and Weiping Li. "On the Adaptive Control of Robot Manipulators". en. In: *The International Journal of Robotics Research* 6.3 (Sept. 1987), pp. 49–59. ISSN: 0278-3649, 1741-3176. DOI: 10.1177/027836498700600303. URL: http://journals.sagepub.com/doi/10.1177/027836498700600303 (visited on 04/29/2024).

[60] Erwin Susanto, Agung Surya Wibowo, and Elvandry Ghiffary Rachman. "Fuzzy Swing Up Control and Optimal State Feedback Stabilization for Self-Erecting Inverted Pendulum". In: *IEEE Access* 8 (2020). Conference Name: IEEE Access, pp. 6496–6504. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2019.2963399. URL: https://ieeexplore.ieee.org/document/8947974/?arnumber=8947974 (visited on 07/16/2024).

[61] Gang Tao et al. *Adaptive Control of Systems with Actuator Failures*. Springer London, June 2013. ISBN: 978-1-4471-3758-0.

[62] TU Diep Cong Thanh and Kyoung Kwan Ahn. "Nonlinear PID control to improve the control performance of 2 axes pneumatic artificial muscle manipulator using neural network". In: *Mechatronics* 16.9 (Nov. 2006), pp. 577–587. ISSN: 0957-4158. DOI: 10.1016/j.mechatronics.2006.03.011. URL: https://www.sciencedirect.com/science/article/pii/S0957415806000523 (visited on 05/23/2024).

[63] Agus Trisanto et al. "Application of Neural Networks to MRAC for the Nonlinear Magnetic Levitation System". en. In: *JSME International Journal Series C* 49.4 (2006), pp. 1073–1083. ISSN: 1344-7653, 1347-538X. DOI: 10.1299/jsmec.49.1073. URL: http://www.jstage.jst.go.jp/article/jsmec/49/4/49_4_1073/_article (visited on 06/19/2024).

[64] Jia-Jun Wang. "Simulation studies of inverted pendulum based on PID controllers". en. In: *Simulation Modelling Practice and Theory* 19.1 (Jan. 2011), pp. 440–449. ISSN: 1569190X. DOI: 10.1016/j.simpat.2010.08.003. URL: https://linkinghub.elsevier.com/retrieve/pii/S1569190X10001796 (visited on 02/12/2024).

[65] H. Philip Whitaker and Allen Kezer. *Design of Model Reference Adaptive Control Systems for Aircraft*. en. M.I.T. Instrumentation Laboratory, 1958.

[66] HP Whitaker. "An adaptive system for control of the dynamics performances of aircraft and spacecraft". In: *Inst. Aeronatical Sciences* (1959), pp. 59–100.

[67] T. Yamada and T. Yabuta. "Neural network controller using autotuning method for nonlinear functions". en. In: *IEEE Transactions on Neural Networks* 3.4 (July 1992), pp. 595–601. ISSN: 10459227. DOI: 10.1109/72.143373. URL: http://ieeexplore.ieee.org/document/143373/ (visited on 06/28/2024).

[68] Yuichi Yoshida and Takeru Miyato. *Spectral Norm Regularization for Improving the Generalizability of Deep Learning*. 2017. arXiv: 1705.10941 [stat.ML]. URL: https://arxiv.org/abs/1705.10941.